

口令與憑據

主篇把憑據放在「站穩」那一章裏，寫到八處存放位置、三種形態、四級範圍就收了，每一條都只給一句判斷依據。本篇是把那一節展開：同一個環節裏的機制、前提、操作順序、失敗模式與痕跡，逐條寫開。

口令與憑據是兩件事，放在一篇裏講，是因為它們在時間上首尾相接：猜口令是為了拿到第一份憑據，而拿到憑據之後做的事，又是拿它去換更多的憑據。兩者的分別在偵測上更清楚。猜口令留下的是失敗的記錄，憑據的使用留下的是成功的記錄，防守方看這兩類記錄的規則、資料源與閾值都不一樣。所以本篇前半講猜，後半講取與用。

本篇是主篇《跟着紅隊走一遍護網演習》的專題篇之一。主篇講整條路的先後次序，專題篇講單個環節的內部。兩篇的關係是骨頭與關節：主篇說「先打點、再站穩」，本篇說「站穩之後的那幾個小時具體在做甚麼」。需要前後文的時候，請回主篇 [跟着紅隊走一遍護網演習](#) 對照着看。

還有一句必須寫在前面。本篇寫的每一個動作，都以拿到**授權**為前提。在演習之外的環境裏，對別人的系統做同樣的事，性質是刑事犯罪。下面所有「怎麼做」，都只在授權範圍之內成立。

1 | 猜口令的三種手法

口令噴灑、**撞庫**、**憑據填充** 這三個詞經常被放在一起講，有些材料乾脆把它們合併成一條「弱口令攻擊」。這對防守方是有害的。三件事在流量上長得完全不同，用一條規則去蓋，結果是每一件都抓不牢。這一章把它們分開寫，每一件都寫清三件事：它的流量特徵、防守方為甚麼要單獨為它寫一條規則、以及紅隊在知道那條規則的前提下怎麼收着自己的動作。

1.1 | 先數清楚哪些地方可以猜

猜口令的第一步不是造字典，是把入口清單數清楚。入口沒數清就開始猜，等於把有限的嘗試次數分給了錯誤的目標。要數的入口有四類。

1. 對外身分入口：虛擬專用網、郵件系統、辦公自動化系統、單一登入、以及對外的堡壘機與運維平台。這一類從互聯網就可以到達，也是防守方最可能已經加上**多因素認證**的一類。

2. 內部身分入口：網域控制站自己、內部應用的登入頁、資料庫、以及各種管理後台。這一類要有一個[立足點](#)才夠得着，而它們通常沒有多因素認證。
3. 非互動入口：郵件協議、檔案共享、遠端管理協定、以及各種把自己的認證轉發給目錄服務的裝置。這一類的特點是沒有網頁界面，防守方如果只監控網頁登入，就會整片漏掉。
4. 設備與應用自帶的帳號：網路設備、安全設備、中介軟體與監控系統。這一類的口令往往根本沒有人改過，屬於後面「默認憑據」那一章。

清單列出來之後，每一個入口要問三件事：它用哪一套帳號體系（目錄服務、應用自建、還是本機帳號）、它有沒有[帳號鎖定](#)策略、它的登入日誌由誰收走。第三問看起來與攻擊無關，實際上決定了紅隊能在這個入口上花多少嘗試次數。一個入口如果日誌只留在本機、而且保留期只有七天，那麼它對紅隊的約束就只剩下鎖定策略一條。

還有一件要在這一步做完的事，是把入口按「同一套帳號體系」分組。同一個目錄服務背後的十個應用，共用同一份口令與同一套鎖定計數；而應用自建的帳號庫與它無關。分組的意義在下一節會看到：鎖定是按帳號算的，但每個入口的失敗計數並不總是在同一個地方累加。

1.2 | 口令噴灑

口令噴灑是用少量常見口令去試大量帳號，每個帳號只試一兩次。它與逐個帳號窮舉的**暴力破解**是兩個方向：暴力破解把嘗試次數集中投在一個帳號上，噴灑把嘗試次數攤薄到很多帳號上。前者一定會撞上鎖定策略，後者的整個設計意圖就是繞開鎖定策略。MITRE ATT&CK 把這種做法單列成一個子技術，¹ 因此它是一個有明確名字、也有明確偵測方法的動作。

它挑的口令有五類。第一類是產品與角色相關的基礎口令，把常見的幾個換成大寫或加一個符號；第二類是單位自己相關的口令，例如單位名稱的縮寫加年份、域名的主體部分加符號；第三類是季節與年份，因為不少單位的口令輪換週期是一季或半年；第四類是符合當前口令策略的最小複雜度組合，例如首字母大寫的單詞後面接三個數字與一個符號；第五類是本地化過的鍵盤路徑與常見詞的變形。這五類的共同點是不針對任何具體的人，所以一份清單可以拿去噴整個單位。

噴灑的流量特徵可以壓成四個「少」與四個「多」。來源少：往往就是一台或幾台機器，因為噴灑不需要分佈式，它本身就不會觸發按帳號計數的

¹見 [T1110.003 Password Spraying](#)；同族的猜口令子技術還有 T1110.001 與 T1110.004。

鎖定。每個帳號的失敗次數少：一次，最多兩次。成功數少：命中率通常在千分之幾的量級，一次噴灑能進去的帳號是個位數。時間窗口短：如果沒有刻意放慢，幾千個帳號的嘗試在十幾分鐘內就結束了。對應的四個「多」是：涉及的帳號多、同一時間窗內的失敗多、重複使用的口令多、請求的形態多。最後一條容易被忽略：同一個工具發出的請求，在標頭、連線參數、請求順序上高度一致，而正常使用者的登入請求形態雜亂得多。

噴灑有幾種常見的變體，它們針對的正是最容易寫出來的那條偵測規則。慢速噴灑把同一個單位的嘗試攤到幾天裏，每天只碰幾十個帳號；來源輪換用住宅代理、雲主機或已控的跳板，讓來源維度上的計數永遠不高；協議輪換同一組帳號分別去試網頁、郵件協議、目錄服務的埠，因為防守方常常只監控了其中一個；時間選擇把嘗試放在工作日的工作時間，理由是這個時段的失敗登入本來就多，使用者打錯口令、口令剛過期、裝置保存了舊口令，都會產生失敗記錄。最值得注意的是最後一種：口令集中過期的那幾天，整個單位的失敗登入量會自然抬高一個數量級，那幾天是噴灑最好的掩護。

防守方為甚麼要為噴灑單獨寫一條規則，而不是和撞庫合用一條「登入失敗過多」。原因在於兩者的失敗分佈完全相反。噴灑的失敗集中在少數來

源上，每個帳號只失敗一次；撞庫的失敗分散在大量來源上，而且會產生大量「帳號不存在」。一條按帳號失敗次數計數的規則抓不到噴灑，因為每個帳號的計數都是一；一條按來源計數的規則抓不到低速撞庫，因為來源太分散。合併成一條，閾值為了壓住誤報必須放寬，放寬之後兩邊都抓不住。所以正確的做法是把兩個維度都做成規則：帳號維度看「同一帳號在短窗口內失敗幾次」，來源維度看「同一來源在短窗口內碰了幾個不同帳號」。

紅隊怎麼規避。規避的目標是把每一步都壓在閾值下面，同時讓自己的流量看起來像這一天裏本來就會有的流量。可以調的四個量是：每個來源每小時碰多少個帳號、每個帳號總共碰幾次、兩次碰同一個帳號之間隔多久、以及失敗的形態像不像正常人。經驗值是每個來源每小時不超過十個帳號，每個帳號在整個行動裏不超過三次，並且這三次要分佈在**帳號鎖定**的計數重置窗口之外。在形態上，同一個來源的請求要錯開時間，不要用固定的間隔，也不要一次把清單跑完；能混進正常流量的做法是跟着目標的通勤時間走，早上八點到十點、下午一點到三點，這兩個時段的登入量最大。

噴灑最怕的不是偵測，是三種自己造成的失敗。一是噴到蜜罐帳號。防守方會放一批從來不用的帳號，它們存在的唯一目的就是被碰。任何一次對它們的登入嘗試都是告警，而且這條告警的優先級很高，因為正常情況下沒

有人會用到這些帳號。二是噴到停用的帳號。停用帳號只會返回一種結果，大量相同的結果本身就是特徵，而且它會讓紅隊的清單迅速變髒：一批帳號全部無效，後面的噴灑就白費了。三是鎖了服務帳號。服務帳號背後往往有服務在跑，它一被鎖，依賴它的業務立刻報錯，值守的人會在幾分鐘內收到工單，這是紅隊最不希望驚動防守方的方式。所以噴灑的清單要先過濾：去掉明顯的服務帳號名，去掉已經確認停用的帳號，並且把優先級最高的幾十個帳號挑出來單獨處理。

1.3 | 撞庫

撞庫是拿別處洩漏的帳號口令組合來試目標系統。它的前提假設很簡單：人在多個系統上重複使用同一套口令。這個假設在互聯網服務上被反覆驗證過，在單位內部也成立，只是規模小一些：員工的企業信箱口令、外部論壇口令、購物網站口令之間，重複的比例相當高。它與噴灑的關係是互補的：噴灑靠「猜到一組常見口令」，撞庫靠「別人已經替我們猜過了」。

撞庫的流量特徵與噴灑正好相反，可以壓成四個「多」與兩個「不存在」。來源多：因為要躲避按來源計數的封鎖，撞庫幾乎一定要分佈式，用代理池或大量被控主機。帳號多：一條洩漏記錄就是一個帳號。每個帳號的口令不同：這是它與噴灑最本質的差別，噴灑是很多帳號共用一個口令，撞

庫是每個帳號各有一個口令。每個帳號的失敗次數少：也是一到兩次。兩個「不存在」是它獨有的特徵。大量帳號在目標系統上不存在，因為洩漏憑據庫來自別的網站，同一個人在目標單位可能根本沒有帳號；大量口令在目標單位的策略下不可能合法，因為洩漏憑據庫是按別的網站的規則生成的，長度與字元集都可能不符合目標的要求。後者會讓請求在進入認證之前就被前端擋掉，在前端留下的是校驗失敗而不是認證失敗，這兩種記錄通常落在不同的日誌裏。

撞庫的成功率比一般人想像的低。公開討論裏的命中率通常在千分之一到百分之一之間，而且隨着目標單位的口令策略與洩漏憑據庫的新鮮度變化很大。但它的絕對收益取決於投入量：一次完整的撞庫可以在幾個小時內跑完幾百萬條記錄。對紅隊來說，這個數字的意義是：撞庫是一條「用基礎設施換結果」的路，它比拼的不是技巧，是代理池的規模與洩漏憑據庫的質量。演習裏的性價比因此不高，除非目標單位有大量對外服務的員工帳號，而且這些帳號在互聯網上有過洩漏。

為甚麼防守方的撞庫規則要與噴灑分開寫。兩條規則的資料源與閾值都不同。噴灑的關鍵訊號是「同一來源、多個帳號、短窗口」，它可以在認證日誌上獨立成立。撞庫的關鍵訊號是「帳號不存在的比例異常」與「同一來

源的嘗試速率異常」，前者要求日誌裏記錄了認證結果的細分原因，後者要求能把來源聚合成代理池級別的實體。這兩個資料條件與噴灑那條規則不一樣，所以它們是兩條獨立的規則，而不是同一條規則的兩個閾值。把兩者合成「登入失敗過多」，會同時失去「每個帳號只失敗一次」與「帳號不存在比例」這兩個最有價值的訊號。

紅隊怎麼規避。撞庫的規避重點在「讓自己在來源維度上不像一個人」，以及在「帳號存在性」這個訊號上做減法。來源維度上的做法是輪換與限速：每個來源只跑一小段就換，並且讓每個來源的請求速率落在正常使用者的範圍內。帳號存在性的做法是預過濾：把洩漏憑據庫裏不屬於目標單位的域名全部去掉，只留與目標郵箱域名一致的記錄；再按目標的口令策略把不可能的記錄剔掉，例如策略要求十二位以上、而洩漏憑據庫裏一半是八位，那就不必去試。這兩步能把「帳號不存在」與「口令格式不合規」這兩個訊號壓下去一大截。

撞庫還有一個不常被提到的自傷。洩漏憑據庫裏的記錄往往包含一批已經被別人用過的組合，其中有些在目標系統上早就被鎖定了。拿這些記錄去撞，會直接產生一批**帳號鎖定**事件，而鎖定事件是防守方優先級最高的告警

之一。所以撞庫之前要用一份「已經失敗過的組合」做去重，這份清單在團隊內部是共享的，它同時也是紅隊自己的成績記錄。

1.4 | 憑據填充

憑據填充 是拿自己在本次行動中取得的憑據去試其他系統。它與撞庫的差別在憑據的來源，與噴灑的差別在它不是猜：手裏的東西已經是對的，問題只是它對哪些地方有效。它是整個內網階段的主要動力，因為一次成功的填充就把一台機器的存取權變成一批機器的存取權，收益遠高於在同一個系統上繼續找漏洞。

填充的流量特徵與前兩者完全不同，它最大的特點是「成功」。前兩者的規則都建立在失敗記錄上，而填充的絕大多數請求會成功登入，所以那些規則對它幾乎不起作用。要抓它必須換一套資料模型：看的是「誰從哪裏登入到哪裏」，而不是「誰失敗了幾次」。具體要看的量有四個。同一個帳號在短時間內出現在多少台不同機器上；同一台來源機器在短時間內向多少台目標機器發起認證；一個「帳號加來源」的組合是不是第一次出現；以及這次登入之後緊接着發生了甚麼動作。最後一條是把兩個層面的日誌接起來的關鍵，單看認證日誌，一次新的來源組合只是一個可疑事件；接上主機日誌

之後，如果它後面跟着建立服務、建立計劃任務、或者讀取本機憑據，它就變成了一條完整的攻擊鏈。

為甚麼這條規則必須獨立於前兩條。前兩條規則的資料源是認證日誌裡的失敗記錄，這一條的資料源是成功記錄加上主機行為日誌，兩者連收集管道都不同。多數單位把失敗記錄收得很好，因為它有明確的閾值；而成功記錄的體量是失敗記錄的幾十倍，很多單位根本沒有把它集中起來，或者集中了但沒有做聚合。這正好是紅隊最喜歡的狀態：動作完全在「合法」的形狀裏，而防守方沒有能力把合法動作之間的關係看出來。

紅隊怎麼規避。填充要控制的不是次數，是關係。同一個帳號的使用要分散在不同時間、不同來源、並且盡量與該帳號原本就會出現的地方重合。例如一個運維帳號本來就會登入管理網段的機器，那麼用它去登入管理網段的另一台機器，在關係圖上並不突兀；而用它去登入一台辦公終端，即使只有一次，也是一條以前從未出現過的邊。所以要規避的不是「用了幾次」，是「用出了一條新的邊」。具體做法有三條：優先沿既有關係走，先查清楚目標帳號平常從哪裏登入到哪裏（AD 的登入工作站限制、使用者對象上的資訊、以及上一台機器上已有的記錄都能給出這張圖）；新邊只走一次，需

要重複訪問的機器改用在台機器上另取一份本機憑據；以及把動作放在該帳號的正常活動時間裏。

填充還有一類容易被自己人打斷的情形。同一支隊伍裏幾個人各自拿到了憑據，如果沒有人做記錄，同一份憑據會被不同的人重複使用，同一台機器會被不同的路徑反覆登入。這種重複在防守方的聚合規則下就是最明顯的特徵：一個帳號在一個下午從五個來源、用五種工具登入同一台機器。所以隊內一定要有一份共享的憑據台賬，記清楚每一份憑據的來源、已經在哪裏用過、以及誰在用。這件事與分數無關，但它決定了這條路能走多遠。

1.5 | 三者的共同約束：噪音預算

三種手法都要在同一個預算裏花錢。噪音預算 是紅隊在演習裏真正稀缺的東西：每一次失敗的登入、每一次異常的來源、每一次被鎖定的帳號，都在消耗防守方的注意力，也在消耗紅隊自己在目標看來「像不像正常流量」的額度。預算花完的標誌不是分數掉下來，是防守方開始改口令、加封鎖、並且開始找人。到那一步，紅隊手裏已有的憑據會在幾十分鐘內失效。

分配預算的順序建議是先取後猜。拿到一份高範圍的憑據，比猜中十個普通帳號更有價值，而且取憑據的動作在日誌上比大批量的失敗登入乾淨得多。只有在取憑據這條路暫時走不通的時候，才把預算轉回猜。

2 | 字典

字典是猜這條路的彈藥。[口令噴灑](#) 直接吃字典，[暴力破解](#) 也吃字典，[撞庫](#) 雖然用的是洩漏憑據庫，但從裏面挑哪一批、按甚麼順序去試，仍然要按目標的[口令策略](#) 來決定。一份按目標訂製的字典與一份通用字典，命中率可以差一兩個數量級。原因不複雜：通用的幾百個口令，單位的口令策略多半早就把它們屏蔽了；而由本單位的人名、單位名與年份拼出來的口令，既符合策略的各項要求，又不會出現在任何通用字典的前幾萬名裏。

2.1 | 取材：公開資訊裏有甚麼

造字典的原材料全部來自公開資訊與已控機器上的本地資訊。公開資訊的部分在第一輪[信息收集](#) 裏已經收過一遍，造字典是把那一遍的產出按口令的形狀重新排一次。要分成六類取。

1. 姓名。來源包括單位的官網人員名單、招聘訊息、招標與中標公告、論文與專利的作者欄、對外的新聞稿、公眾號與影片號的署名、以及公開的

代碼倉庫裏的提交記錄。取的不是「有哪些人」，而是「名字怎麼拼」：全拼、姓氏加名首字母、姓首字母加名全拼、英文名、以及這幾種的大小寫寫法。這一類決定字典能不能對準具體的人。

2. 工號與員工編號。來源是洩漏憑據庫、內部文件的頁眉頁腳、以及各種登記表的檔名。工號的價值在於它常被直接當成口令的一部分，而且它是有規律的數字，可以從已知的幾個推出一整段區間。
3. 單位名稱與域名。包括全稱、簡稱、英文名、拼音首字母、商標名、以及域名的各段。這是所有口令裏出現頻率最高的一類詞根，因為它對員工來說最好記。
4. 年份與季節。包括當前年份與前兩年的年份、季度、以及單位的年度口號。這一類存在的理由只有一個：口令輪換。每輪一次，就有一批人的新口令等於舊口令換一個年份。
5. 業務與產品名。單位對外提供的服務名稱、內部系統的代號、以及常見的合作方名稱。這一類命中的往往是服務帳號與共用帳號，因為這類帳號的口令通常由項目組隨手取。
6. 歷史口令的模式。如果目標單位或它的合作方在過去有過洩漏，那麼這批記錄裏的口令不只是可以直接拿去撞庫的憑據，還是一份「這個單位的員工怎麼取口令」的樣本。從中提取的是規則而不是口令本身：常用詞根有

哪些、年份放在前面還是後面、用甚麼符號、大小寫怎麼變。規則比口令值錢，因為規則可以外推到今年。

取材階段的產出是三張表：詞根表、數字表、連接符與符號表。後面所有的生成都是在這三張表上做乘法。

2.2 | 拼接規則與鍵盤路徑

拼接規則是「詞根加數字加符號」的各種排列。把最常見的十幾種寫出來，覆蓋的面已經很大：詞根加年份、詞根加年份加符號、詞根加生日、詞根加手機尾號、詞根加工號、姓加名首字母加年份、單位縮寫加詞根加數字、以及詞根重複兩遍。這些規則之所以有效，是因為它們對人來說最好記，而口令策略的複雜度要求剛好可以用「加一個符號、加幾個數字、首字母大寫」這三個動作滿足。

字符替換是拼接的第二步。常見的替換是母音或字母換成形近符號與數字：字母 a 換成 @、字母 o 換成數字 0、字母 i 或 l 換成數字 1、字母 s 換成 \$ 或數字 5、字母 e 換成數字 3、字母 g 換成數字 9、字母 t 換成數字 7。替換要按位元組合生成，不能只生成全替換與不替換兩種，因為實際的口令多半只換了其中一兩個字母。最後再決定大小寫：首字母大寫、全小寫、以

及每個單詞首字母大寫。到這一步，一個詞根會膨脹成幾百個候選，所以字典的大小不是問題，排序才是問題。

鍵盤路徑是一類獨立的口令來源，它與人名地名無關，所以不會被「不含常見詞」這類策略攔住。常見的路徑有三類。一類是同排相鄰鍵的橫向序列，例如字母鍵盤左上到右下的斜線、以及中間一排的橫向序列。一類是三個豎列的縱向序列，例如左列由上到下、以及它在數字排上的對應。一類是包含數字與符號的組合路徑，例如數字排按住換檔之後的符號序列。這三類路徑的長度通常在六到十二之間，雙向都要生成，並且要把起點在鍵盤上平移幾個位置，因為每個人的習慣起點不同。

還有一類是「策略驅動」的構造，它不來自任何具體的詞，而是來自口令策略的形狀。例如策略要求至少八位、包含大寫、小寫、數字、符號各一，那麼最小合法口令就是「一個大寫字母加一個小寫字母加一個數字加一個符號加四個任意字元」。這類口令在單位裏的比例往往被低估：員工為了應付策略，會取一個剛好合法的長度，然後把需要的四類各放一個，其餘用最省事的鍵盤路徑補齊。這種形狀可以用生成規則直接枚舉，不需要任何詞根。

2.3 | 從頁面文案與報錯訊息反推口令策略

目標的口令策略決定了甚麼樣的口令有可能存在，所以反推它是造字典的一步，不是可選項。反推的資料源按可靠性從高到低排，有四個。

1. 前端代碼。這是可靠性最高的一處。註冊頁與改密頁的校驗幾乎都在前端做一遍，所以最短長度、最大長度、必須包含的字元類別、不得包含使用者名、不得與最近若干次口令重複，這些規則就在頁面的腳本、表單元素的屬性、以及多語言資源檔裏。單頁應用的打包檔案裏還常能找到完整的正規表示式。拿到這一段，等於拿到了策略的全文。
2. 報錯訊息。後端返回的錯誤碼比提示文案細。同一個「口令不符合要求」，在有的系統上會細分成「太短」、「缺少數字」、「包含使用者名」幾種。把這些訊息收集起來，可以逐條確認策略的每一項。要注意的是錯誤訊息的顆粒度本身也是資訊：如果所有失敗都返回同一句話，說明目標刻意做了統一，這時候只能靠長度邊界去二分。
3. 長度邊界試驗。在授權範圍內的測試帳號上，用一串逐漸加長的字元去試改密介面，直到被接受為止，最短長度就定了。上限也同樣重要：有系統仍然限制在十四位或二十位，上限低意味着字典可以少生成一大段。
4. 已控機器上的本地策略。這一處只有在拿到立足點之後才有，但它給出的資料最準。在 Windows 上，網域的口令與鎖定策略可以從任何一台已加

入網域的機器上讀出來；在本機層面，還有本機策略與組策略的落地檔案。在 Linux 上，對應的是登入定義檔、口令質量設定檔，以及認證模組的設定。讀出來的東西包括最短長度、複雜度要求、最長使用期限、歷史口令數，以及在下一章要用的鎖定參數。

反推的時候有三個邊界情形要記住。第一，策略變更通常不回溯。單位把最短長度從八位提到十二位之後，老帳號只要不改口令就一直合法，所以「當前策略很嚴」不等於「所有帳號的口令都很長」。第二，不同系統的策略不一致。對外入口、內部應用、網路設備各有各的規則，單位的資訊安全管理制度寫的是最嚴的那一份，實際執行的往往是每個系統各管各的。第三，服務帳號與共用帳號通常不走互動式改密，它們的口令往往是當初為了「能跑起來」而設的，形狀與員工口令完全不同，用員工字典去猜這類帳號效率很低。

2.4 | 輪換規律怎麼推

輪換規律推得準，字典就能從「幾十萬條」壓到「幾十條」。推的依據有四個來源。

1. 公開時間軸。單位一般在幾個固定節點要求改密：年度資訊安全檢查前、上級單位的專項檢查前、以及內部制度規定的週期。這些節點對外是有痕

跡的，例如檢查通知、安全月活動、以及新聞稿。把這些時間點記下來，就能猜到下一輪甚麼時候來。

2. 技術痕跡。目錄服務裏每個帳號都有一個「口令最後設定時間」的屬性，任何一個已通過認證的帳號都能查。把一個部門的帳號排在一起看，如果它們的時間集中在某幾天，說明單位在集中強制改密；如果時間散落好幾年，說明輪換制度根本沒有執行。單一帳號的這個屬性還有一個用法：知道某人的口令是兩年前設的，那麼它很可能從未被改過，也就更可能出現在舊洩漏裏。
3. 登入失敗的節律。口令到期前後，使用者會先用舊口令試一次、失敗、然後才去改。所以在集中改密的窗口前後，整個單位的失敗登入量會出現一個小高峰。把這個節律記下來，既可以用來推算下一次窗口，也可以在下一章講的噴灑裏當掩護。
4. 口令的形狀本身。一條舊口令就是一組規則的樣本。把洩漏憑據庫裏屬於這個單位的一批口令拿出來看，數字放在末尾還是開頭、年份是四位還是兩位、符號是感嘆號還是點，這些會形成很強的習慣。按這套習慣生成新口令，命中率遠高於通用規則。

輪換規律最有價值的一種形態是「遞增」。如果一批舊口令的最後幾位是年份或季度，而單位又確實按週期強制改密，那麼新口令幾乎必然是同一

批詞根換一個年份。這意味着拿到任何一份舊憑據之後，不需要重新猜，只需要試有限的幾個變體。這條路在下一節講憑據的時候還會用到，因為它讓一份過期的憑據重新變得有用。

也要接受一種結果：目標單位根本不輪換。近年主流的口令指引已經不再推薦強制定期更換，理由是頻繁更換會促使人們選更弱、更有規律的口令。² 所以推不出輪換規律是正常的，這時候字典的年份維度就按當前年份與前一年生成，不必往前推太多。

2.5 | 排序、分批與失效

字典生成出來之後有三件事要做，它們決定了這份字典能不能真的用上。

第一是排序。嘗試次數是稀缺資源，所以字典的順序就是優先級的順序。排在最前的是「最有針對性且最可能一次命中」的那些：單位縮寫加當前年份、姓名全拼加當前年份、姓名加常用數字、以及最常見的幾組基礎口令。排在最後的是由鍵盤路徑與純隨機生成的部分。排序還要按帳號分組：針對張三的字典應該排在張三那一組的最前面，而不是和所有人的字典混在一起。

²見 [NIST SP 800-63B-4](#)，該文本對「任意週期強制更換口令」的立場與此一致。

第二是分批。分批的依據是**帳號鎖定**策略與防守方的告警閾值，不是字典的長度。如果目標的鎖定閾值是五次，計數重置窗口是三十分鐘，那麼一個帳號在一個行動週期裏的安全預算大約是三次。三次就是三批，批與批之間至少要隔開一個計數重置窗口。分批還有一個副作用是好的：每一批跑完之後可以根據結果調整下一批，例如命中的口令都帶同一個後綴，那麼下一批就把這個後綴的權重提高。

第三是失效管理。字典是有保鮮期的。目標在演習期間改口令、補丁、或者把你已經猜到的那幾個口令作廢，這份字典裏相應的部分就報廢了。更常見的失效是策略變更：單位在演習開始前剛把最短長度提高，於是你按舊策略生成的一大段全部不可能存在。所以字典要帶版本，每一次按新資訊更新都要留一份記錄，否則會出現「上一輪已經確認錯誤的候選又被拿出來試」這種浪費，而這種浪費消耗的是最貴的預算。

3 | 帳號鎖定

帳號鎖定是猜口令這條路上最主要的限制。它不是防禦得最好的那一環，而是最容易算清楚的一環。算清楚之後，紅隊能把嘗試次數精確地分配出去；算不清楚，就會出現兩種壞結果：要麼過於保守、把預算浪費在沒有

嘗試的目標上，要麼把一個高價值帳號試到鎖定，既暴露了自己，也影響了目標的業務。

3.1 | 先判斷目標有沒有鎖定

判斷分被動與主動兩條路。被動那條在拿到立足點之後才走，主動那條在最外面就能做，但兩者要配合使用，因為主動測試本身就在消耗注意力。

被動判斷是直接讀目標的策略。在 Windows 的網域環境裏，一台已加入網域的機器上可以讀到網域的口令與鎖定策略，要看的四個參數是：鎖定閾值、鎖定持續時間、計數重置窗口、以及是否對內建的系統管理員帳號也計數。在 Linux 上，對應的資訊在認證模組的設定與帳號資料庫裏，讀出來的是失敗次數上限、鎖定時間，以及是否按來源而不是按帳號計數。被動判斷的優點是不產生任何額外的失敗記錄，缺點是它只反映某一套帳號體系的策略，而目標對外的那幾個入口可能各自有自己的防護。

主動判斷要做的是把三種防護區分開：帳號鎖定、來源封鎖與限速、以及人機驗證。這三種在表面上的表現很像，都是一串失敗之後不再讓你試，但它們對紅隊的意義完全不同：帳號鎖定意味着這個帳號的預算是有限的，來源封鎖意味着換一個來源就能接着試，人機驗證意味着自動化必須付出額

外成本。區分的方法是設計一組最小的對照試驗，用低價值或不存在的帳號，並且嚴格控制次數。試驗的邏輯有四步。第一步用一個不存在的帳號連續失敗，如果它也被「鎖定」，說明攔下來的是來源或前端，而不是帳號；第二步用一個存在的低價值帳號失敗到臨界值，然後用正確的口令再試一次，如果正確口令也被拒，帳號鎖定成立；第三步換一個來源再試同一個帳號，如果仍然被拒，確認是按帳號鎖定而不是按來源；第四步記下恢復的時間，這個時間就是鎖定持續時間。

主動試驗本身是有成本的，上面四步至少要產生十幾次失敗登入，如果目標單位把「同一個帳號連續失敗」直接做成告警，那麼這次試驗就已經把紅隊的位置暴露給值守的人了。所以要做兩件事降成本。一是把試驗放在整個行動最早的時段，那時候暴露的代價最小，因為手裏還沒有值得保護的通道；二是把試驗的失敗混進一個本來就會失敗的場景裏，例如在口令過期窗口期做，或者在目標自己的一次變更之後做。

還有一個容易被忽略的邊界：同一個目錄服務的不同入口，失敗計數不一定累加在同一個地方。按帳號計數的實作中，計數值保存在處理那次認證的伺服器上，而不是每一台都同步同一份計數。於是同一組憑據分別打到不同的伺服器，每一台看到的失敗次數都可能低於閾值，而帳號始終沒有被

鎖。這個特性在微軟的公開文件裏沒有被寫成一條明確的規則，但它是由「計數保存在處理認證的伺服器上」這個實作決定的，實際效果取決於網站的部署方式與複寫間隔。對紅隊的意義是：把嘗試分散到不同的認證伺服器上，是在不提高總次數的前提下所能做的最有效的規避。對防守方的意義是：只在其中一台伺服器上寫鎖定規則，會讓這條路完全隱形。

3.2 | 每個帳號的嘗試次數預算

預算是這樣算的。先取鎖定閾值，減去一個安全邊際，再減去這個帳號在本次行動中已經消耗的次數。安全邊際是為了兩件事留的：目標單位的正常使用者可能也用錯口令，把這個帳號的計數往上抬；以及紅隊自己可能在某個入口上重複觸發了一次。閾值已知時，安全邊際取一；閾值未知時，整個預算就取二到三次，這是一個在多數系統上都不會觸發鎖定的上限。

計數的兩個性質決定了預算怎麼花。第一，一次成功的認證會把失敗計數清零。這意味着預算不是整個行動的總額，而是一個會刷新的額度：只要中途有一次成功，後面又可以再花。紅隊可以用這一點，把高價值帳號的嘗試安排在已經確認能成功的地方之後。第二，計數只在重置窗口內累加。如果重置窗口是三十分鐘，那麼每隔三十一分鐘失敗一次，無論多少次都不會

鎖定。這條性質讓「慢」變成了一種資源：對那些值得長期盯住的帳號，可以按窗口的節律每輪只試一次，用時間換次數。

預算還要按帳號分級，不能所有帳號用同一個數。建議分三檔。第一檔是一般員工帳號，預算可以接近閾值減一，因為猜錯的代價低，而且這類帳號本來就經常失敗。第二檔是管理員帳號與各類高權限帳號，預算為零到一，也就是不猜、或者只在有很強的先驗理由時猜一次。第三檔是服務帳號，預算為零。理由在下一節：服務帳號一旦被鎖，被影響的是業務，而業務中斷比一次告警嚴重得多。

最後要記住，鎖定策略不是唯一的限制。防守方還可以在自己的監控上設一個比鎖定閾值更低的告警線，例如「同一帳號失敗三次」就直接打電話找人核實。鎖定閾值可以從設定裏讀出來，告警線讀不出來，只能從防守方的反應速度上推。所以真正的預算是兩者取小，而後者未知。保守的做法是按二到三次執行，這一條在演習裏幾乎不會錯。

3.3 | 把管理員帳號試到鎖定的後果

這件事在演習裏的性質與技術上的後果是兩回事，兩件都要算。技術上的後果比較直接。帳號在鎖定持續時間內完全不可用，即使口令是對的；已

經拿到的這個帳號的憑據同時失效，不管是明文還是雜湊；如果它是服務帳號，依賴它的服務會立刻報錯或停止，業務側可能在幾分鐘內就有人看到。鎖定這件事本身還會產生一條明確的事件記錄，而這條記錄在防守方那裏是優先級最高的幾類之一，因為正常使用者很少真的把自己鎖掉。如果單位把鎖定的恢復方式配成「需要管理員手動解鎖」，那麼這段不可用時間不是十五分鐘，而是直到有人看到工單為止。

對演習成績的後果更重。**演習規則**一般會把「影響目標業務正常運行」列為違規，鎖掉一個承載業務的管理員帳號正好落在這一條裏。現實中的處置通常有三種：扣分、責令停止使用該手法、或者由裁判判定這次動作不計分並記錄在案。還有一種不寫在規則裏但真實存在的後果：目標單位的值守人員會被驚動，他們會立刻改口令、收緊策略、並且開始排查。紅隊在前幾個小時積累的所有憑據，可能因為這一個動作全部作廢。

有一個特殊情形值得單獨記。內建的系統管理員帳號在多數 Windows 版本上預設不參與鎖定計數，這一項在各家的作業系統安全基準裏都被單獨列出來，要求把它打開。³這對紅隊是一把雙刃劍。好的一面是這個帳號可以無

³見 [Account Lockout Policy](#)；內建系統管理員帳號的鎖定是各家基準單列的一項加固要求，預設不計數。

限次嘗試，所以它是唯一一個「預算不受鎖定限制」的高權限目標；壞的一面是它同時是防守方最關注的帳號，任何針對它的失敗嘗試都會被單獨計數與告警，而且它的名字本身就決定了沒有一個正常使用者會用它登入。所以在實務裏，針對這個帳號的猜測只在兩種情況下做：目標的這個帳號確實沒有改過預設口令，或者紅隊已經從別處拿到了它的口令、只是需要驗證一次。

還有一類後果是針對紅隊自己的。紅隊在行動中往往會用一個自己臨時建立的帳號來跑工具，或者在目標環境裏用一個已拿到的普通帳號做日常操作。如果這個帳號被鎖，紅隊自己的通道就斷了。更隱蔽的一種是：紅隊用同一個帳號在多台機器上試憑據，其中一台機器對這個帳號的失敗計數已經接近閾值，於是紅隊在完全沒有預期的情況下觸發了鎖定，並且誤以為是目標改了口令。所以在憑據台賬裏，每個帳號都應該有一欄記着「已消耗的失敗次數」。

4 | 默認憑據

默認憑據是這條路上成本最低的一種入口，因為它不需要猜。它與前面三種手法的差別就在這裏：噴灑與撞庫是「不知道口令所以試」，默認憑據是「知道口令是甚麼、只是不知道對方有沒有改」。它在演習裏的地位比在互聯

網上高得多，原因是內部設備的默認口令存活率遠高於暴露在互聯網上的設備。

4.1 | 從哪查

查詢的順序與成本成反比，所以要從最便宜的那一頭開始：先知道對面是甚麼產品，再去找那個產品的默認憑據。產品識別靠**指紋**，而默認憑據的來源有六處。

1. 廠商的官方文件。安裝指南與快速入門章節通常直接寫着初始帳號與初始口令，以及首次登入之後是否強制修改。這一處的價值不只是口令本身，還在於它會說明「這個產品在甚麼條件下不會要求改密」，例如以容器方式啟動時，或者以離線方式匯入設定時。
2. 產品的映像與固件。這是可靠性最高的一處。把安裝映像或固件解開，裏面有初始設定檔、初始化腳本、以及資料庫的初始化腳本，默認口令、默認的加密金鑰、默認的管理埠都在裏面。這一處還能拿到「這個版本改了甚麼默認值」這種資訊，而這一類變更通常不會寫進公告。
3. 公開的默認口令清單。社群維護的字典集裏有專門的一類是默認憑據，按產品名與型號組織。這一類清單的用法是按產品查，不是拿去盲掃，因為盲掃會產生大量無意義的失敗請求。

4. 容器與編排的範例檔案。官方映像的說明、容器編排的範例、以及各類部署教學裏的設定片段，常常直接寫着默認的帳號、密碼與連接埠。這一處的命中率高，因為近幾年的部署大量走這條路。
5. 搜尋引擎與代碼託管。把產品名加上「默認口令」、「初始密碼」、「default password」直接搜，再把產品名加上設定檔的常見檔名去代碼託管裏搜，往往能找到別人貼出來的完整設定。這一處要小心的是版本：同一產品的不同版本默認口令不同。
6. 已控機器上的本地資訊。拿到第一台機器之後，那台機器上的運維文件、採購與驗收資料、組態備份、以及運維人員自己的記錄裏，通常寫着整批設備的型號與口令。這是效率最高的一處，因為它一次覆蓋一整批設備，而且它給的是「這一批設備實際上用的是甚麼」，而不是「廠商出廠時是甚麼」。

這六處的順序有一個例外要記住。如果已經拿到了立足點，第六處應該排在最前面，因為從內部文件裏讀到的口令是當前有效的，而從互聯網上查到的口令有一半已經被改過。只有在還沒有立足點的時候，才按前面五處的順序走。

4.2 | 哪些產品最常見

下面按六類列出默認憑據最常見的落點。列舉的重點不是具體口令，而是每一類產品的「默認形態」，因為知道了形態就知道該去試甚麼。

1. 邊界與網路設備。防火牆、虛擬專用網閘道、負載均衡、以及交換機與無線控制器。這一類的默認形態是「一個固定的管理員帳號加一個固定的初始口令」，以及一個對內開放的管理埠與網頁界面；部分型號還保留着只允許從特定網段訪問管理界面的設定，而那個設定在部署時常被改成全部允許。它們是演習裏價值最高的一類，因為一台就等於一個入口。
2. 安全與運維設備。堡壘機、日誌與審計平台、端點防護的管理端、以及各類廠商自帶的運維通道。這一類的默認形態是「管理端的初始口令從來沒有改過」，而它們往往存放着全網的憑據，所以拿到一台的收益遠大於它本身的權限。
3. 中介軟體與應用平台。應用伺服器的管理控制台、持續整合與部署平台、代碼託管平台、以及各類監控與看板系統。這一類的默認形態是「安裝之後直接用」，管理帳號與口令寫在安裝文件裏；更常見的一種形態是「管理界面沒有認證」，因為預設只監聽本機，而部署時為了遠端維護把它改成監聽全部位址。

4. 資料庫與快取。這一類的默認形態不是一組帳號口令，而是「不設口令」或「只允許本機連線」這兩種設定中的一種。不設口令的實例只要連接埠可達就直接可用；而只允許本機連線的設定，在有立足點之後可以通過本機轉發繞過。還有一種形態是應用自己的連接字串裏帶着高權限帳號，那屬於組態檔那一類憑據。
5. 監控與日誌系統。這一類的默認形態是「匿名可讀」與「默認帳號可寫」。以日誌與指標系統為例，不少實例在初始狀態下允許未認證的查詢，而寫入與管理介面用默認帳號；前者已經能洩漏大量內部資訊，包括主機名、內部位址與服務清單。
6. 辦公與終端外設。印表機與影印機、網路儲存、視訊會議終端、以及各類監控攝像與錄影設備。這一類的默認形態是「網頁管理界面加固定帳號」，以及各版本之間長期沿用的匿名訪問設定。它們的價值不在算力，而在於它們處於辦公網裏，是進入辦公終端的跳板。

還有一類不屬於上面任何一類，但它出現的頻率最高：網路管理協議的共同體字串。它與帳號口令無關，默認值只有幾個常見的字串，而只要它是默認值，就可以用只讀權限把整台設備的拓撲、介面、以及路由資訊全部讀出來。這一條在[信息收集](#)階段就能用，不需要任何認證。

4.3 | 為甚麼這條路在演習裏更好用，以及它的注意事項

在互聯網上，默認憑據的收益隨着暴露時間迅速下降，因為暴露在互聯網上的設備會被持續掃描，任何一個默認口令都會在幾天內被人改掉或被人用掉。在演習的目標裏情況相反。靶標單位的內部設備通常不對外，它們從來沒有被互聯網上的人碰過，所以廠商出廠的默認口令可能已經用了十年。同時，演習目標裏的管理網段常常沒有被完整清點，紅隊能拿着一份默認口令清單去試，而防守方手裏甚至沒有這批設備的名單。這兩個原因合起來，讓默認憑據成為演習裏確定性最高的一種入口。

它也有三條注意事項。第一，拿到設備之後不要改它的口令。這一類設備往往不在單位的變更流程裏，改了之後可能沒有人能恢復，業務影響會直接體現在演習扣分上。第二，不要把默認憑據用在公共目標上。公共目標池裏的機器同時被多支隊伍盯着，你先用默認口令進去，下一支隊伍可能就把口令改掉了，或者防守方已經因為別人的動作把它加固了。第三，用默認憑據進去的證據要留全。默認憑據的動作看起來太正常，以至於很容易被人懷疑「這是不是你自己配的」，所以要保留登入前的設備資訊、登入時的認證記錄、以及能證明操作發生在演習時間窗內的東西。

5 | 多因素認證

多因素認證 是猜口令這條路的天花板。在它覆蓋到的地方，猜到口令不再等於登入成功。但它不是一個開關，而是一組分級的機制：不同種類的第二因素，對紅隊的阻力差別很大，而覆蓋率在不同的入口之間也從來不是一致的。這一章分四步寫：怎麼判斷一個入口有沒有、口令那條路被截斷到甚麼程度、兩類繞法的原理與前提、以及繞完之後還剩下甚麼限制。

5.1 | 先判斷某個入口的覆蓋率

覆蓋率的判斷要有清單，因為「單位部署了多因素認證」與「這個入口要求多因素認證」是兩件事。判斷的角度有五個，按可靠性從高到低排。

1. 從登入流程看。在最外面就能做的一件事是走一遍登入流程：輸入口令之後是否必然出現第二個步驟。必然出現、且沒有跳過的方法，說明這個入口覆蓋了。這裡有兩個要看的細節：有沒有「記住這台裝置」或「信任此瀏覽器」這類選項，因為它意味著有條件豁免；以及第二個步驟失敗之後能不能後退重試，因為有些實作會把整個會話作廢。
2. 從協議端點看。同一套帳號體系背後的網頁入口可能覆蓋了，而它的舊協定端點沒有。郵件與檔案共享類的舊協定在多數實作上不支援互動式的第二因素，所以只要那個端點還開放，帳號口令就可以單獨完成認證。這是

一個結構性的缺口，與單位部署得多認真無關，它取決於有沒有把舊協定關掉。

3. 從應用密碼一類的機制看。對那些不支援第二因素的客戶端，有的系統提供一種「應用專用口令」，它是一個長期有效的獨立口令，登入時不需要第二個因素。只要這個機制開着，它就等於在覆蓋範圍上開了一扇後門。
4. 從設定看。拿到立足點之後，可以讀到單一登入的設定、條件訪問類的策略、以及每個應用自己的認證設定。這裡能看到的關鍵資訊是「例外」：哪些來源位址、哪些服務帳號、哪些內部網段被排除在第二因素之外。例外是覆蓋率判斷裏最重要的部分，因為它通常不寫在制度檔裏，只寫在設定裏。
5. 從行為看。同一批帳號在不同入口的登入結果會暴露不一致。如果一個帳號在網頁入口需要第二因素、而在舊協定端點不需要，那麼用同一個帳號分別試兩個入口，結果的差異就是答案。這種比較本身要消耗嘗試次數，所以要與上一章的預算一起算。

把五個角度合起來，結論通常不是「覆蓋」或「沒覆蓋」，而是「主入口覆蓋、有幾個結構性缺口」。對紅隊的用處也就落在這裏：缺口的位置決定了投入的方向。如果對外入口全部覆蓋、而且第二因素都是抗釣魚的種類，那麼繼續在口令上投入的收益就很低，應該把資源移回[社工釣魚](#)與對外服務

漏洞那兩條路。如果覆蓋率高但存在舊協定端點或服務帳號豁免，那麼口令這條路仍然值得走，只是要走那幾個缺口。

5.2 | 口令那條路被截斷到甚麼程度

截斷的程度取決於第二因素綁定了甚麼。把常見的幾種排在一起看，差別很清楚。簡訊與語音[一次性動態碼](#)綁定的是手機號碼，而號碼是可以被轉移或攔截的，所以它擋住的是「拿着口令從別處登入」，擋不住「同時控制號碼」的情形。應用產生的一次性動態碼綁定的是那把種子，它的弱點在於種子可以被釣魚頁面即時取走，因為在使用者輸入的那一刻它就在攻擊者手裏。推播確認綁定的是使用者手上的那台裝置，它的弱點在使用者本身：反覆推播會讓人疲勞，而人會在疲勞中按同意，這類做法被單列成一個技術條目。⁴ [硬體金鑰](#)綁定的是「來源」，它在認證時把當前的網域名稱放進簽名裏，所以釣魚頁面拿到的簽名在真實站點上用不了。⁵這一種是目前公認難繞的一類。

所以判斷「這個入口的多因素認證擋不擋得住」，不能只看有沒有，要看是那一種。判斷的方法還是看互動形態：需要使用者輸入六位數字的是動態

⁴見 [T1621 Multi-Factor Authentication Request Generation](#)。

⁵見 [T1539 Steal Web Session Cookie](#)，該條目的緩解一節說明了硬體金鑰與受管裝置對會話重放的抑制作用。

碼一類；需要使用者拿手機點同意的是推播一類；需要插入裝置或在作業系統層面確認的是硬體金鑰一類。這三種在頁面上留下的請求路徑也不同，從瀏覽器的請求位址就能看出來。

還有一點要說清楚：多因素認證截斷的是「猜」，不是「用」。如果紅隊手裏已經有一份有效的會話憑證，或者有一台已經通過認證的機器，那麼第二因素在那一份憑證的有效期內沒有任何作用。這正是下一章要講的事：口令這條路的出口被堵住之後，憑據那條路的價值就上升了。

5.3 | 第一類繞法：讓驗證通過的那一刻發生在攻擊者手裏

這一類的做法是**中間人釣魚**。攻擊者不去破解第二個因素，而是在受害者與真實網站之間放一個透明的代理：受害者看到的頁面是真的（內容由真實網站即時取回），協定也是真的（憑證往真實網站送），所以第二個因素的驗證會真實地通過一次。攻擊者要的東西在這次通過的結果裏：認證完成之後網站發給瀏覽器的那個**會話憑證**。拿到它之後，攻擊者不再需要口令，也不再需要第二個因素，直接以已認證的身分訪問即可。

它的原理可以壓成一句：它繞過的不是第二個因素，而是「第二個因素與後續訪問之間的綁定」。如果這個綁定不存在，也就是網站發的會話憑證是一個任何地方都能用的記名票，那麼一次通過就等於一張長期通行證。

它的前提有四條，缺一條就做不成。第一，使用者要在一個不屬於真實網域的頁面上輸入口令。這一條要靠[社工釣魚](#)那一篇的辦法解決，與本篇無關，但它同時也說明這一類做法不是純技術問題。第二，代理要能通過目標站點的反自動化檢查。現代的身分提供者會看裝置指紋、看傳輸層的參數、看請求的順序，也會在回應裏放需要執行的腳本，代理必須把這些都原樣轉發過去，任何一項處理得不對，流程就會在第二步中斷。第三，目標發出的會話憑證必須是可轉移的。如果憑證與來源位址綁定，或者與裝置綁定，那麼從攻擊者的位址拿去用會立刻失效。第四，這次認證的結果要有足夠長的有效期。會話憑證過期之後，攻擊者手裏的東西就作廢了，要重新釣一次。

這四條前提不成立的時候，失敗的方式也很典型。如果第二因素綁定了來源，代理拿到的簽名在真實站點上驗不過，受害者的認證也會失敗，這種失敗在防守方那裏表現為「一次異常的認證錯誤」，而不是一次成功的登入，所以它反而更容易被注意到。如果目標要求受管裝置才能訪問，那麼用竊取

的會話憑證從一台非受管裝置訪問時，會被要求重新認證，竊取來的東西在這一步作廢。

這一類做法在日誌上的樣子值得單獨記。在身分提供者那一側，記錄是一次成功的、來自一個新的地理位置或新的裝置的登入，中間沒有任何失敗，也沒有一段明顯的攻擊流量。在使用者那一側，他會看到一個自己沒有印象的登入，或者一個已經在別處登入的提示。也就是說，這一類做法把偵測的責任從「認證伺服器」推到了「行為基線與使用者回報」上，而後者恰恰是多數單位最薄弱的環節。

5.4 | 第二類繞法：偽造一份已經通過的憑證

這一類做法的對象是驗證的那一端。在聯邦認證與票據認證這兩類機制裏，有一個共同結構：驗證的一方不自己檢查口令，它只檢查一條憑證上的簽名，而簽名來自某個它信任的簽發端。於是只要紅隊拿到簽發端用來簽名的那把金鑰，就可以自己造一條憑證，宣告任意一個身分已經通過了認證。整個口令與第二因素的環節被從中間跳過去，一次都沒有發生。

這個結構有三個常見的實例。第一個是聯邦認證的斷言簽名憑證被拿走，攻擊者用它簽出任意使用者的斷言，於是可以在多個依賴這一套認證的

雲服務之間自由移動，這條路在公開通告裏有完整描述。⁶第二個是票據簽發帳號的雜湊被拿走，攻擊者用它簽出任意使用者的票據，這一條在域環境裏是最有名的做法之一。⁷第三個是內部的憑證頒發機構私鑰被拿走，攻擊者可以簽出任意主體的**身分憑證**，而依賴這套憑證的系統會把它當成合法身分。

它的前提比第一類更硬，因為它要求先拿到簽發端本身。具體要滿足三件事。第一，簽名用的金鑰要能取出來。有的部署把金鑰放在硬體裏，導不出來，這條路就直接斷了；有的部署把金鑰匯出權限留在伺服器上，那麼拿到那台伺服器的管理權就等於拿到金鑰。第二，要知道驗證那一端的信任設定。憑證裏有簽發者標識、主體的名稱欄位、有效時間窗、以及受眾，這些欄位填錯，驗證端會直接拒絕，而且拒絕的理由通常不會回給紅隊，所以要靠試錯或靠事先讀到的設定。第三，時間要對得上。這一類憑證都帶有有效時間窗，如果紅隊控制的機器的時間與目標差得太多，造出來的憑證在驗證時會因為時間不在窗內而被拒。

這一類做法的位置決定了它與本篇的關係。它的前提全部落在內網與域這一層，也就是說，紅隊要先在域裏拿到足夠的權限，才能走到簽發端。所

⁶見 [CISA AA20-352A](#)。

⁷見 [T1558 Steal or Forge Kerberos Tickets](#)。

以它的展開不在本篇，而是在第五篇《域與內網橫向》。本篇寫到這裏為止，只把它的原理與前提交代清楚，說明為甚麼在口令這條路已經被完全堵住的時候，這條路仍然存在。

5.5 | 繞完之後還剩甚麼限制

兩類繞法都解決了「怎麼進去」，沒有解決「進去之後能做多少」。剩下的限制有四條。第一是條件訪問類的策略。即使身分已經確認，訪問的範圍仍然可能被裝置合規狀態、來源網段、或者應用的敏感性限制住，所以竊取一個會話憑證能拿到的東西，不一定等於那個使用者的全部權限。第二是有效期。會話憑證與票據都有期限，而兩類繞法拿到手的東西都不會自動續期，過期之後要重新做一次。第三是行為基線。這一類登入在認證日誌上是完全成功的記錄，唯一的異常在位置、裝置與時間上，而這三樣恰好是可以被基線模型看出來的。第四是使用者自己。被釣的使用者知道自己在甚麼時候登入過，一次自己沒有印象的登入很容易被上報，而上報之後防守方會直接作廢所有會話。

這四條合起來的實務含義是：繞過多因素認證拿到的是一個窗口，不是一個立足點。要在這個窗口裏完成的事情必須事先排好，而且要有一個萬一窗口關閉時的退路。把「繞過」當成終點，是這一類做法最常見的失敗原因。

6 | 憑據的取得

猜到口令是為了拿到第一份憑據，而憑據的價值在於它會被反覆使用。這一章講兩件事：在一台已經拿下的機器上，憑據分別存在哪八處、每一處怎麼取；以及取出來的東西有三種形態，每一種能用在甚麼場合。取憑據在攻擊鏈上的位置是提權之後的第一個動作，因為它是唯一一個「一次投入、多次產出」的環節。

6.1 | 本機八處存放位置與取法

八處存放位置按取的順序排列，順序的依據是「收益除以成本」。前兩處收益最高、成本中等；中間三處收益中等、成本很低；後三處收益取決於運氣，但只要機器上有人用過，命中率就不低。

6.1.1 | 第一處：程序的記憶體

這是收益最高的一處，因為它存的是可以直接使用的東西：互動式登入留下的口令材料、雜湊、以及票據，都在作業系統負責認證的那個程序的記憶體裏。除此以外，瀏覽器、資料庫客戶端、遠端桌面客戶端、命令列的雲服務工具，也各自在記憶體裏存着已經通過認證的憑證。

取法是做一次**記憶體轉儲**，再從裏面把憑證結構解析出來。需要的權限是本機管理員，因為讀取別的程序的記憶體需要一個專門的權限。如果有核心層的能力，可以繞過使用者層的保護，但那要加載驅動，而加載驅動本身是防守方重點監控的行為。

這一處的痕跡最重，也是防守方唯一有成熟規則擋住的一處。作業系統與端點防護都會記錄「有程序去讀認證程序的記憶體」，這件事在正常運維裏極少發生。應對的手段有三類：一是把認證程序設成受保護的處理程序，讓非特許的程序連開句柄都做不到；二是用虛擬化把憑證換成不可直接使用形式，讓記憶體裏根本沒有明文與雜湊；三是限制除錯權限的授予範圍。所以這一處在實務上的判斷順序是：先試一次，如果被擋住就不要再試第二次，直接轉向第三處與第四處。

6.1.2 | 第二處：本機的帳號資料庫

這一處存的是**口令雜湊**，分兩類。一類是這台機器自己的本機帳號，它們的雜湊在帳號資料庫檔案裏；另一類是**快取憑據**，也就是這台機器替網域帳號保存下來的登入材料，它存在的理由是讓使用者在網域控制站不可達時仍能登入，預設保存的數量是十個。兩類的價值不同：本機帳號的雜湊只對這台機器有效，而快取的網域帳號材料可以用來推導那個使用者的口令，推出來之後範圍就是整個網域。

取法上有兩個技術要點。第一，執行中的系統會鎖住帳號資料庫檔案，所以不能直接複製，要用影子副本、從系統登錄區匯出、或者從原始磁碟讀。這幾個做法各自會留下不同的痕跡：影子副本會產生建立快照的事件，登錄區匯出會在系統目錄裏留下檔案。第二，拿到的東西是雜湊，能不能用取決於演算法的強度與口令的熵，詳細判斷見本章第三節。在 Linux 上這一處對應的是口令檔與金鑰檔，包括使用者目錄下的私鑰、允許登入的公鑰清單、以及提權設定。這一類憑據的取法比 Windows 簡單，通常只要讀檔案權限，而痕跡也更小。

6.1.3 | 第三處：票據與會話的存放區

這一處存的是「已經通過認證」的結果，不是認證材料本身。它包括作業系統層面的身分票據、應用層面的登入工作階段、以及各類命令列工具與容器編排工具的快取權杖。它的特點是不需要推導、不需要離線運算，拿出來就能用。代價是它會過期。

取法分兩層。系統層面的票據與第一處在同一塊記憶體裏，所以取的動作與第一處相同，防護也相同。應用層面的東西要簡單得多：它們大多落在使用者目錄下的檔案裏，有的直接是明文，有的用與使用者綁定的方式加密，而以該使用者身分執行的任何程序都能解密，因為解密要的那把鑰匙就在同一台機器上、綁在同一個使用者身上。所以這一層幾乎不需要管理員權

限，只需要以那個使用者的身分執行程式碼。這一條在實務上很重要：很多紅隊把注意力全放在管理員權限上，而普通的員工終端上就有大量應用層憑據，其中不少能直接登入內部系統。

6.1.4 | 第四處：組態檔與部署腳本

這是成本最低、命中率最高的一處。運維人員把口令寫在檔案裏是常態，因為「能跑起來」比「不被看到」優先級高。要找的檔案有幾類：應用自己的設定檔（連接字串、快取位址、第三方介面的金鑰）；容器編排與部署的設定檔；自動化運維的變數檔；各類腳本（部署腳本、備份腳本、巡檢腳本）；以及備份與歷史檔（帶備份副檔名的舊版本、打包的壓縮檔）。

搜尋的關鍵在於「搜內容而不只是搜檔名」。檔名可以記住幾十個常見的，但真正命中的往往是命名毫無規律的檔案。有效的做法是兩步：第一步按檔名粗篩出設定檔與腳本；第二步在這些檔案裏按關鍵字搜，關鍵字要覆蓋中英文與縮寫，並且要注意編碼差異，因為有的設定檔是寬字元編碼，按普通編碼去搜會一個字都搜不到。還要注意大檔案與二進位檔案會被跳過，而備份與資料庫傾印恰好是大檔案，所以要單獨處理。

這一處有一個容易被忽略的邊界：從設定檔裏讀到的憑據多半是服務帳號，而服務帳號的範圍大、輪換少，所以它比同樣是「明文口令」的員工帳

號值錢得多。另一個邊界是檔案權限，有的設定檔只有服務自己讀得到，那麼以一般使用者身分去讀會失敗，要換到服務的身分或者用管理員權限。

6.1.5 | 第五處：瀏覽器

瀏覽器裏有兩種東西。一種是它替使用者保存的帳號與口令，另一種是已經登入的工作階段與登入狀態。前者的價值在於它往往是使用者自己常用的那幾套系統，而人在不同系統上重複使用口令的習慣，正好讓這一份清單可以用在別處；後者的價值在於它不需要口令，直接就是一個已認證的身分，這與第二類繞過多因素認證的做法是同一件事。

取法上，以該使用者身分執行的程式碼可以直接拿到保存的內容，因為保護它們的機制綁定的是使用者身分，不是硬體。瀏覽器廠商近年加了更強的保護，把解密限制在瀏覽器自己的程序裏，讓外部程序拿着檔案也解不開，這一條改變了這一處的成本，所以在動手之前要先確認目標瀏覽器的版本與保護狀態。工作階段那一部分只要讀設定檔目錄裏的儲存區即可，有些還能在記憶體裏直接取。痕跡集中在「有程序去讀瀏覽器的設定檔目錄」這件事上，以及取走之後的使用行為。

6.1.6 | 第六處：運維與管理工具

這一處的價值在於它常常是「一批機器的憑據」，而不是一台機器的。為了免密操作，遠端桌面與終端工具會保存連線資訊與口令，檔案傳輸工具會保存站點與口令，資料庫客戶端會保存連線設定，虛擬化與儲存的管理端會保存整個資源池的帳號，組態管理與自動化工具會保存「能管所有機器」的帳號，備份軟體會保存「能讀所有資料」的帳號。拿到其中任何一個，下一步就不是一台機器，而是一片機器。

取法與第四處類似，都是找使用者目錄與程式目錄下的設定檔，區別是這一類檔案的位置比較固定，而內容常帶着明文或可還原的編碼。要記住的邊界有兩個。一是這些工具裏的憑據往往有很強的權限，但它的使用在目標側會呈現為「這個工具在跑」，所以用它的時候要沿用工具自己的腳本或介面，而不是拿着它去手工登入一堆機器，後者在日誌上是一串突兀的新連線。二是有的工具把憑據交給作業系統的憑據管理員保管，這一種要從憑據管理員那一側取，而它與使用者身分綁定，所以要在那個使用者的身分下取。

6.1.7 | 第七處：程式碼倉庫

這一處有兩個特點。一是憑據即使已經被刪掉，在歷史記錄裏仍然在，因為版本控制保存的是完整的變更歷史；二是它保存的憑據通常是「開發與

部署用」的，而這一類憑據的權限往往比生產環境的運維帳號更大，因為它要能部署。

取法是在機器上找版本控制的目錄，然後在完整歷史裏按關鍵字搜，看的不是當前的檔案內容，而是每一次變更裏新增的那些行。除了歷史，還有幾處固定位置：版本控制工具的憑據檔、會話與存取權杖的快取、以及倉庫自身的設定檔裏記着的遠端位址與存取方式。還有一類「倉庫」不在這台機器上，而在內部的套件伺服器與映像倉庫上，它們的存取權杖一般也保存在設定檔裏。

這一處的邊界是「取得到但用得上未必」。歷史裏的憑據有一部分已經被輪換，有一部分對應的帳號已經停用，所以在用之前要先判斷它是否還有帳號與權限對應。判斷的方法是把候選憑據按「帳號是否存在」與「權限是否還在」兩步過一遍，這兩步本身要用掉嘗試次數，所以要與上一章的預算放在一起算。

6.1.8 | 第八處：自動建置與部署系統

這一處是整條鏈上權限最集中的地方。持續整合與部署系統存在的意義就是「能往所有環境部署」，所以它保存的憑據覆蓋開發、測試、生產的全部環境；它保存的雲端部署身分能創建與刪除資源；它保存的容器編排權杖

能操作整個叢集；它的產物倉庫憑據能替換將被部署的產物。拿下一台建置節點，等於拿下它能部署到的所有環境。

取法有兩條。一條是從建置節點本身取，因為節點上一定有它執行任務時要用的憑據，這些憑據或者在使用者目錄下，或者在節點的環境變數與工作目錄裏；另一條是從建置系統的管理界面取，如果有帳號能登入管理界面，憑據清單是可以直接看的（界面上通常不顯示明文，但可以通過修改任務的方式把它印出來，或者直接把任務改成執行自己的指令）。第二條路徑的痕跡集中在管理界面的稽核日誌上，而這一類日誌很多單位根本沒有開，或者開了卻沒有人看。

這一處還有一類衍生目標：建置系統會把部署腳本與設定模板一起放在倉庫裏，而模板裏常常帶着佔位符與真值混在一起，真值會出現在某一次提交、某一個環境變數、或者某一個被建置產物打包進去的檔案裏。所以從建置系統取到的憑據往往不是一份，而是一整批。

6.2 | 三種形態各能用在哪

八處取出來的東西歸成三種形態：明文口令、雜湊值、票據。它們的用途差別很大，所以在拿到之後第一件事是判斷形態，而不是急着用。

1. 明文口令。它是用途最廣的一種：可以做互動式登入、可以用在那些要求明文重放的認證上、可以用在資料庫與網路設備上、也可以在目標的改密入口上把口令改成一個自己知道的值。最後一條是它獨有的能力，也是它與另外兩種形態最本質的差別：只有明文才能改成新的口令，而改密是**權限維持**的一種手段。它的限制有兩條：一是它是三種形態裏最容易被防守方的策略收走的（目標一改口令，它就沒用了）；二是它的每一次使用都會計入**帳號鎖定**的失敗計數，所以用明文去試範圍的時候要按上一章算的預算走。
2. 雜湊值。它能用在以雜湊本身作為認證材料的場合，也就是不需要把口令還原出來就能完成認證的那些協定；它也能在環境允許的情況下換成票據使用；它還能通過**離線破解**還原成明文。離線這一條的意義在於它不產生任何網路流量，所以不受鎖定策略與速率限制，代價是它需要算力，而且能不能還原取決於演算法與口令的熵：同一個弱口令在慢雜湊下仍然很快出來，而一個足夠長的口令在慢雜湊下就是算不出來的。它的限制有三條：不能改密；在目標把舊認證協定關掉的環境裏完全失效；以及它對應的是帳號當前的口令，目標一輪換它就沒用了。
3. 票據。它是三種裏唯一「即取即用」的一種：不需要口令，不需要雜湊，拿着它就能訪問它所代表的那個身分能訪問的一切，這一步在術語裏叫**票據傳遞**。它的範圍與那個身分完全相同，所以在判斷價值的時候，問的

不是「這張票據能做甚麼」，而是「這個帳號能做甚麼」。它的限制就是有效期：過了有效期就必須重新取得，而重新取得的前提是紅隊仍然控制着簽發的來源。它還有兩個不明顯的性質：票據的有效性不依賴口令的當前值，所以目標改口令不會讓它立刻失效；以及它與所在機器的時鐘相關，時鐘偏差過大會直接導致它被拒。

三種形態的取用順序建議是：先看有沒有票據，再看有沒有雜湊，最後才決定要不要花算力去還原明文。這個順序的理由是每一步的成本都比前一步高一個量級，而收益的差別沒有那麼大。只有在需要改密或者需要用到那些只吃明文的系統時，才值得走到第三步。

還有一種形態沒有列在上面，但在近幾年的環境裏越來越重要：身分憑證。它既不是口令也不是雜湊，而是一張由內部頒發機構簽發的證書，用來證明主體的身分。它的特點是可以被直接使用，而且不參與口令輪換，所以在口令這條路被切斷的環境裏，它是一個獨立的出口。它的展開屬於第五篇，本篇只把它記在形態清單的末尾。

6.3 | 取不到的時候

取憑據失敗有四種典型情形，每一種的應對不同，認清情形比反覆嘗試更重要。

1. 權限不足。手裏只有一個普通使用者身分，讀不了帳號資料庫，也讀不了別的程序的記憶體。這時候剩下的路是第四處與第五處：這個使用者自己的設定檔、自己的瀏覽器、以及他有權讀的那部分目錄。接下來再走提權那一篇。
2. 認證程序的記憶體被保護。這是端點防護成熟單位的常態。被擋住的時候不要在原路徑上換工具重試，因為每一次重試都是一條新的告警。應該轉向「不需要在本機取」的路：從設定檔取、從網路層取（認證轉發與中間人），以及從別的機器取同一個帳號的憑據。同一批機器裏總有防護較弱的一台。
3. 拿到的東西形態不對。常見的是拿到了雜湊，而需要的系統只吃明文；或者拿到了票據，而目標機器不在同一個信任範圍裏。這一種的判斷要靠事先把目標系統的清單與它們接受的認證方式對上，而不是拿到之後才試。
4. 憑據的範圍是空的。取到了憑據，但那個帳號已經停用、已經被移出群組、或者那台機器已經下線。這一種最容易浪費時間，因為它看起來像

「試不進去」，而實際上不是技術問題。避免的方法是每拿到一份憑據就先判斷它的範圍，判斷方法見下一章。

7 | 憑據的範圍與時效

一份憑據拿到手之後，第一件事不是用它，而是判斷它值不值得用。判斷有兩個維度。一個是範圍，也就是它能在多少台機器、多少個系統上成立；另一個是時效，也就是它還能成立多久。兩個維度都要在它之前有結論，否則會出現兩種浪費：把高價值的憑據花在低價值的目標上，或者把已經失效的憑據反覆拿出來試，而每一次嘗試都在消耗**噪音預算**。

7.1 | 四級範圍

範圍可以按四個層級分：**本機帳號**、**本機管理員**、**服務帳號**、以及**域帳號**。這四級不是憑據的形態之分，而是它所代表的身分的權限之分。同一個層級裏，明文、雜湊與票據的範圍是一樣的，差別只在能不能用與怎麼用。

7.1.1 | 第一級：本機帳號

本機帳號只存在於這一台機器上，它的範圍就是這一台。判斷方法很直接：帳號名不帶網域前綴，在本機的帳號清單裏能找到。它的用處是讀取這

台機器上更多東西，以及作為一個不起眼的落腳身分；它對[橫向移動](#)沒有幫助，因為別的機器上沒有這個帳號。這一級最容易被高估。紅隊在一台機器上取到一組本機帳號與口令，有時會以為它是一個突破口，實際上它連「下一台機器」都到不了。

7.1.2 | 第二級：本機管理員

本機管理員的範圍是「所有共用同一組本機管理員口令的機器」。這一批有多大，取決於部署方式，差別可以很大。逐台手工安裝的環境裏，每一台機器的本機管理員口令都不一樣，這一批就只有一台；用統一映像批量部署、或者用[組態管理工具](#)統一下發本機管理員口令的環境裏，這一批可能覆蓋辦公網的全部終端。

判斷這一批有多大，有兩個方法。一個是比較：在已經拿到的兩台機器上，比較同一個本機帳號的雜湊，如果相同，說明它們是同一批；拿到的機器越多，這張圖越準。另一個是從部署方式推：看機器上有沒有統一的分發代理、看系統映像的來源、看本機帳號的建立時間是不是同一天。兩個方法合起來，一般能判斷出這一批是「幾台」還是「幾百台」。

這一級的用途是快速鋪開，代價是動靜大。用同一個本機管理員口令去登入一批機器，在防守方的聚合規則下表現為「同一個帳號在同一時段出現

在大量機器上」，這是最容易被看出來的一種橫向。所以合理的用法是：只在這一批裏挑少數幾台關鍵的，並且把登入時間拉開。

7.1.3 | 第三級：服務帳號

服務帳號是為某個服務配置的帳號，它的範圍是那個服務被授權訪問的一切。它在四級裏的性價比最高，原因有三個。第一，它很少被輪換，因為改了它就會斷服務，而改一個員工的口令只需要通知本人。第二，它的權限通常大於實際需要，配置的時候為了「確保服務能跑起來」，往往給了比最小權限更多的東西。第三，它的位置通常在關鍵路徑上，例如備份、監控、部署、資料庫服務，拿到它就等於拿到了一整片。

判斷一個帳號是不是服務帳號，可以從三處看。一是看它有沒有服務主體名稱，有服務主體名稱的帳號是被某個服務註冊在目錄裏的；二是看它在哪些機器上有「以服務身分登入」的權限；三是看它的口令形狀，服務帳號的口令往往很長很隨機，因為它是程式生成的、不需要人記。第三條有一個例外要記住：十幾年前配的服務帳號，口令往往是短而可記的，而且它可能從來沒有被改過。

服務帳號還有一類特殊形態：以內建服務身分運行的服務。它們不在目錄裏，也就沒有口令可以取，但它們在網路上的認證會用到機器帳號，所以

紅隊從這一類服務上拿到的是機器帳號的憑據，而機器帳號的範圍是這台機器本身加上它在域裏被授予的權限。這一條在第五篇會展開。

7.1.4 | 第四級：域帳號

域帳號的範圍是整個**域與域控**所管轄的範圍，具體能到哪一步取決於它屬於哪些群組。它是最有價值的一級，也是痕跡最重的一級。每一次用它做認證，都會在處理認證的伺服器上留下一條記錄，而這些記錄在多數單位裏是被集中收集的。所以域帳號的使用有一個明確的取捨：它的收益最大，但它的每一次使用都在告訴防守方「有人在用這個帳號」。

這一級內部還要再分。普通域帳號能做的有限：讀共享目錄、查詢目錄資訊、訪問它自己有權限的內部應用；但它的價值在於它是一張「合法的存在證明」，在目錄查詢這一類動作上比任何工具都自然。管理類域帳號能做的就完全不同：能登入一大批伺服器、能讀域內的全部帳號資訊、能操作域控。兩者之間的距離，通常就是**提權**那一篇講的內容。

四級範圍的判斷要在「用它之前」完成，而完成的方式本身也要盡量低噪音。低噪音的判斷方法是查而不是試：拿到一個帳號之後，先以它查詢自己能看到的目錄資訊（自己是誰、屬於哪些群組、能登入哪些機器），再從

這些資訊推出它的範圍；而不是拿起它就登入一堆機器，用成功與失敗去試出範圍。後者會產生大量登入事件，而且其中失敗的部分還會計入鎖定計數。

判斷完成之後，一份憑據應該能被歸到這四級中的一級，並且帶上一句話的說明：它能在哪一片上成立。這句話是後面循環的基礎。

7.2 | 票據的有效期與續期

票據這一種形態多了一個別的形態沒有的性質：它自己會續。在域環境的預設設定下，使用者票據的有效期是十小時，而續期窗口是七天；服務票據的有效期也是十小時。⁸ 這三個數字放在一起，對紅隊的意義是：在拿到一張票據之後，只要在七天內讓它續期，就可以把它一直維持到演習結束，而不需要重新拿到口令或雜湊。

票據續期 這件事有兩個要點。第一，續期不需要口令，它用的是票據自己帶的材料，所以目標在演習期間改了那個帳號的口令，並不會讓這條路立刻斷掉。第二，續期會產生週期性的事件記錄，也就是說，一個已經被防守方盯上的帳號，它的票據續期會在固定的間隔上留下痕跡。對紅隊來說，這

⁸見 [Kerberos Policy](#)，這一組設定決定票據與續期的最大期限，各單位的實際值以讀到的策略為準。

意味着長期靠續期維持一張票據是有代價的；對防守方來說，這是一條可以寫成規則的訊號：一個不應該有互動式登入的帳號，卻在規律地續期。

票據還有三個容易踩到的邊界。一是時鐘：票據的有效性判斷依賴雙方時間，偏差超過容許值就會被直接拒絕，所以在用票據之前要先確認目標機器的時間。二是信任範圍：一張票據只在簽發它的那個域及其信任範圍內有效，換到不信任的域就用不了。三是帳號狀態：帳號被停用之後，已經簽發出去的票據在有效期內仍然可以使用，這一條常被忽略，因為人們會以為「停用了就進不去了」。對紅隊的含義是拿到票據之後要盡快用；對防守方的含義是停用帳號之後還要作廢它已經發出去的憑證。

7.3 | 口令輪換與帳號被鎖

時效的另一半是失效。憑據失效有三種原因，它們對紅隊的影響不一樣，排期時要分別對待。

1. 口令輪換。它讓明文與雜湊失效，因為這兩種形態對應的是帳號當前的口令。它不影響有效期內的票據，也不影響那些以自己的方式保存身分的應用工作階段。輪換對紅隊最有意義的不是失效本身，而是它的時機：如果目標在演習中途集中改密，基本可以斷定洩漏已經被發現，紅隊此時應該停止使用手上的憑據，而不是加緊使用。還有一種相反的情形：目標按固

定週期輪換，而輪換規律已經被推出來，那麼一份舊憑據可以用來生成新口令，這一段的辦法在前面講字典的那一章。

2. 帳號被鎖。它讓憑據暫時失效，即使口令是對的也用不了。恢復時間取決於鎖定持續時間，或者取決於有沒有管理員手動解鎖。這一種失效對紅隊的影響最大，因為它同時說明了一件事：有人在對這個帳號做猜測，而且已經超過了閾值。如果是紅隊自己造成的，正確的反應是停止、記錄、並且在台賬上把這個帳號標成不可用。
3. 帳號被停用。它讓憑據永久失效，而且會讓已經簽發的憑據在過期之後無法續期。停用通常意味着防守方已經確認了這個帳號被利用，這是最嚴重的一種訊號，出現它就應該按「已經被發現」來處理整個行動。

把範圍與時效合起來看，一份憑據的價值可以寫成一個簡單的判斷：範圍越大、剩餘有效期越長、失效的可見度越低，它就越值錢。四級範圍裏，服務帳號在後兩項上都佔優；三種形態裏，票據在第一項上不佔優但在第二項上佔優。紅隊在排期的時候，應該把「剩餘有效期短的東西」排在前面用掉，把「不會被輪換的東西」留到關鍵時刻。

8 | 憑據的循環

憑據這條路的核心不是「拿到一份憑據」，而是一個會自己加速的循環：拿一份憑據去試更多機器，在能進去的機器上再取新的憑據，新的憑據又能試更多機器。這個循環的速度遠快於逐台找漏洞，因為它不需要等待漏洞利用的條件成熟，只需要一次認證。這一章講三件事：循環是怎麼轉起來的、怎麼控制它的速度與噪音、以及甚麼時候應該停下來。

8.1 | 循環是怎麼轉起來的

循環有四步。試，也就是判斷這份憑據能在哪裏成立；進，也就是在它成立的地方取得訪問權；判，也就是在進去之後判斷這一台的價值；再試，也就是在這一台上取新的憑據，回到第一步。四步裏最容易做錯的是第一步與第四步：第一步被跳到直接用，第四步被誤以為是「順手看看」。

循環會加速，但不是每一輪都快。決定速度的是能不能找到「跨批次」的憑據。如果每一輪取到的都是本機帳號，那這個循環就是平的一輪一台，永遠鋪不開；如果某一輪取到了共用同一口令的本機管理員，循環就從一台跳到一批；如果取到了服務帳號或者組態管理工具保存的憑據，循環就直接跳到了整片網路。所以循環的關鍵動作是刻意去找那幾類憑據，而不是把每

一台機器都翻一遍。實務上的順序建議是：在每一台新進去的機器上，先看它的本機管理員口令是不是同一批、再看它上面有沒有運維工具與部署腳本、最後才去看它自己的本機帳號。

循環有兩個瓶頸，它們決定了這個循環能轉多久。第一個瓶頸是嘗試次數，受帳號鎖定與防守方的告警線限制；第二個瓶頸是取憑據的成功率，受端點防護的限制。兩個瓶頸的性質不同：第一個是「不能多做」，第二個是「做了也沒用」。認清是哪一個卡住了，決定了下一步是降速還是換路。

8.2 | 速度與噪音怎麼控制

控制的核心是一個取捨：循環轉得快，覆蓋的機器就多，但被聚合規則看出來的機率也高；轉得慢，在演習的時間窗內可能到不了目標。這個取捨沒有通用答案，它取決於目標單位的監控強度，而監控強度可以從前面幾步的觀察裏估出來：失敗登入多了以後有沒有來源被封、有沒有帳號被鎖、有沒有出現明顯的排查動作。

可以調的四個量。一個是每份憑據的使用次數。建議按範圍分：高價值的憑據只用一到兩次，而且用在最關鍵的機器上；低價值的憑據可以多用，因為它的失效不心疼。一個是登入之間的間隔。同一個帳號的兩次登入之間

要有間隔，而且要避開目標的運維高峰與備份時段，因為那些時段的登入量本來就大，但也有它們自己的規律，紅隊的動作混進去之後一樣可以被看出來。一個是來源的分佈。盡量沿既有關係走，也就是用那些本來就會從這個來源登入到這個目標的帳號。一個是目標的挑選。先做「可能共用口令」的機器群，因為那裏的成功率高，一次嘗試就能確認一整批；後做需要單獨突破的機器，因為那些嘗試的成功率低而噪音高。

優先做「只認證、不落地」的一輪。這一輪的目的是把可達範圍畫出來：只完成認證，不在目標機器上執行額外的程序，也不寫檔案。這一輪的噪音最小，因為它留下的只是一次登入；而它給出的資訊最多，因為它直接回答了「哪些機器在這份憑據的範圍內」。畫完這張圖之後，再挑出少數幾台值得落地的機器去做第四步，這樣可以避免在每一台機器上都留下取憑據的痕跡。

還有一件必須在隊內解決的事，是把憑據與使用情況做成一份共享的台賬。台賬要記四樣東西：每一份憑據的來源、它的範圍、它已經在哪裏用過、以及誰在用。這份台賬的第一個作用是避免重複：同一份憑據被不同的人重複嘗試，或者同一台機器被不同的路徑反覆登入，在防守方的聚合規則下是最明顯的特徵。第二個作用是避免撞車：幾個人同時對同一台高價值機

器動手，產生的登入與取憑據行為會疊在一起，看起來遠比實際規模大。第三個作用與分數有關：提交證據的時候需要說明攻擊路徑，而路徑的起點就是這份台賬。

8.3 | 甚麼時候停下來

循環不會自己停，所以要有明確的停止條件。四種情形出現任何一種就應該停。

1. 收益不再增長。連續幾輪取到的都是同一批口令下的機器，或者都是沒有權限的本機帳號，說明這個循環已經到了它的邊界。這時候正確的動作是換一份憑據，而不是把同一份憑據試更多機器。
2. 預算用完。**噪音預算**是有限額的。當連續幾次動作都需要先判斷「會不會被看到」，就說明預算已經接近見底。見底的標誌是防守方的反應速度變快：前一小時怎麼試都沒事，現在換一個來源就被封。
3. 出現被發現的跡象。三種跡象要立刻停：碰到了**蜜罐**、有帳號被鎖、以及目標開始集中改密。這三種出現的順序通常就是嚴重程度遞增的順序，而第三種出現時，行動已經被確認。
4. 邊際效益。演習是有時間窗的，而憑據的數量是沒有上限的。在剩餘時間不多的時候，繼續擴大一張已經足夠大的圖，不如把現有的成果整理成可以提交的證據。這一條與技術無關，但它決定了最後的分數。

停下來之後手上的東西並不會浪費：已經畫出的可達範圍、已經判斷過範圍與時效的憑據、以及這一路的台賬，正是下一階段要用的東西。下一階段是「憑據在域這一層怎麼用」，也就是本篇結尾指向的那一篇。

9 | 痕跡與偵測

這一章站在防守方那一側看本篇講的所有動作。要回答三個問題：用憑據進去與用漏洞進去，在日誌上到底差在哪；為甚麼前者的告警優先級天然更低；以及防守方應該寫哪幾條規則。最後再加一節，說明這些規則紅隊能繞到甚麼程度、以及為甚麼繞不乾淨。

9.1 | 兩種進去的方式在日誌上差在哪

兩者的差別在兩個層面上，第二個層面比第一個層面重要得多。

第一個層面是內容。用漏洞進去，痕跡集中在主機上發生的事情：一個不該被建立的孩子程序被建立了，父子關係不合理（辦公軟體起了命令列、或服務端腳本宿主起了系統工具）；一個服務被安裝，一條計劃任務被建立；一個可執行檔案被寫到不常寫的目錄；一個程序去讀了另一個不相關程序的記憶體；以及一條到外部的連線。這些都是「動作」，每一條都能單獨判斷。

用憑據進去，痕跡集中在身分上：一個帳號在某個時間從某個來源登入到某台機器。它對應的記錄是登入成功、登入失敗、取得票據、以及被指派特殊權限這幾類。這些記錄裏沒有任何一行是「異常動作」，每一行拿出來看都是系統的正常功能。

第二個層面是資料源。漏洞的痕跡落在主機端，由端點感測器、系統日誌與網路流量記錄；憑據的痕跡落在認證伺服器與被訪問的機器上，由[認證日誌](#)記錄。這兩個資料源在不少單位裏是兩套系統、兩個團隊在管，而認證日誌的體量比主機日誌大得多，所以它常常被縮短保留期，或者根本沒有做[日誌集中](#)。結果是：同一個紅隊，走漏洞那條路會撞上主機側的規則，走憑據那條路可能整條都沒有被記錄下來。

還有一個更根本的差別。漏洞的痕跡是「一個點」，單條記錄本身就足以判斷；憑據的痕跡是「一條關係」，單條記錄幾乎沒有資訊量。一條「某帳號從某位址登入某機器」的記錄，在一個每天有幾十萬條同類記錄的環境裏，靠它自己是判斷不出任何東西的。它只有在被聚合成關係之後才有意義：這個帳號以前從哪些來源登入過、這台機器以前被哪些帳號訪問過、這次登入之後緊接着發生了甚麼。這解釋了為甚麼偵測憑據這條路的規則一定是聚合規則，也解釋了為甚麼它對資料的完整度要求更高。

9.2 | 為甚麼憑據這條路的告警優先級更低

低不是因為它不重要，而是因為訊號雜訊比低，而原因有五個。

1. 正常的同類事件太多。內網裏每天都在發生大量的跨機器登入：運維人員的遠端管理、掃描與盤點、備份與歸檔、監控與巡檢、軟體分發、以及各類服務帳號的週期性動作。這些全是真的，而它們與攻擊者的登入在單條記錄上沒有分別。
2. 缺少**基線**。要判斷「帳號從未在這個來源出現過」，前提是先有一張「帳號與來源的正常關係表」。多數單位沒有這張表，少數有表的單位也沒有把它做成可以自動比對的形式。沒有基線，聚合規則就只能按絕對數量設閾值，而絕對數量的閾值在旺季時全是誤報。
3. 分級規則的慣性。多數單位的告警分級是按事件類型定的：程序行為與失敗認證算高，成功認證算資訊。於是一條成功的、但完全異常的登入，在佇列裏排在一堆失敗認證後面，而值守的人在告警疲勞之下只會看排在前面的。
4. 特徵庫不覆蓋。紅隊在這一條路上用的全是系統自帶的工具與合法協定，沒有任何檔案特徵、沒有任何已知的惡意行為簽章。以特徵比對為主要手段的防護，對這條路的檢出率接近零。

5. 保留期。認證日誌的體量大，有的單位把它的保留期設得比主機日誌短，或者只保留失敗的部分。於是「三天前這個帳號從哪裏登入過」這個問題在需要回答的時候已經沒有資料了。

這五個原因合起來，正好是紅隊偏好這條路的理由。它不是因為紅隊在這一條路上技術更高，而是因為防守方對這一條路的可見度更低。而可見度是可以修的，修它不需要買新設備，需要的是把認證日誌收全、留夠、並且把它與主機日誌關聯起來。

9.3 | 防守方該寫的規則

下面十二條按「投入產出比」排，前六條是必須有的，後六條是有條件就做。每一條都寫清資料源與誤報來源，因為閾值只能按本單位的基線調，照抄別人的數字通常不可用。

1. 同一帳號在短時間內從多台機器登入。資料源是成功登入記錄，聚合維度是帳號，時間窗按本單位的運維節奏定。這一條抓的是憑據填充。主要的誤報來源是運維人員與自動化工具，所以要先為它們建立白名單，而白名單必須定期審，否則紅隊拿到運維帳號之後就直接走進了白名單。
2. 同一來源在短時間內對多個不同帳號失敗。資料源是失敗登入記錄，聚合維度是來源。這一條抓的是口令噴灑。閾值建議先按「每個來源每小

- 時不同帳號數」設，再用兩週的資料把誤報壓下去。誤報主要來自共用出口位址的場景，例如有線與無線共用一個出口、或者有集中代理。
3. 帳號不存在的失敗佔比異常。資料源是失敗登入記錄中區分得出原因的欄位。這一條抓的是撞庫，因為撞庫用的帳號清單來自別處，會有相當比例在目標不存在。如果日誌裏區分不出「帳號不存在」與「口令錯誤」，這條規則寫不了，要先把日誌的欄位補上。
 4. 帳號鎖定事件。這一條單獨列出來，是因為它的優先級應該最高。正常使用者很少把自己鎖掉，而鎖定意味着有人在猜，並且已經猜到了閾值。它的誤報來源是忘了舊口令的人，可以按帳號類型區分：服務帳號與管理帳號被鎖幾乎必然是異常。
 5. 服務帳號出現互動式登入。服務帳號的正常行為是網路登入，不應該出現互動式或遠端互動式的登入。出現就說明有人拿着它的憑據在別的地方登入了。這一條的資料源是成功登入記錄裏的登入類型欄位，誤報很少，值得單獨做成高優先級。
 6. 認證成功之後緊跟着主機動作。這是把低優先級升級為高優先級的那一條關聯規則。要做的是把認證事件與主機事件按帳號與時間關聯：一次成功登入之後的幾分鐘內，同一台機器上出現了服務建立、計劃任務建立、遠端過程呼叫，或者有程序去讀認證程序的記憶體。認證事件單獨

看是資訊級，關聯之後就是一條完整的攻擊鏈。這一條的工程成本最高，因為它要求兩類日誌進同一個平台。

7. 新的帳號與來源組合。資料源是兩份清單：每個帳號歷史上的來源清單、以及每台機器歷史上的帳號清單。出現新組合就報，並且按目標機器的敏感性分級。誤報來自人員調動與設備更換，用「首次出現」加「後續動作」兩個條件可以壓下去。
8. 高權限帳號在非管理位置登入。管理類帳號應該只出現在管理網段與管理主機上。它出現在辦公終端上，無論成功與否都值得看。這一條與上一條常常同時命中，可以合併成一個更高的級別。
9. 不可能的移動與新裝置。針對對外入口與通過多因素認證之後的登入。看的是同一帳號在兩個相距很遠的位置之間的時間差，以及從未出現過的裝置與瀏覽器。這一條是偵測會話竊取的少數有效手段之一。
10. 蜜罐帳號被使用。放一批絕對不會被正常使用的帳號，它們存在的唯一目的就是被碰。任何對它們的登入都是惡意的，而且不需要閾值。這一條的成本極低、訊號極乾淨，是所有單位都應該有的。
11. 認證程序的記憶體被讀取。這一條屬於端點側，但它必須與上面的規則一起寫，因為它抓的是「取憑據」這一步，而上面十一條抓的是「用憑據」那一步。兩步都抓，紅隊才沒有可以選擇的便宜路徑。

12. 票據續期的異常節律。針對那些不應該有互動式登入、卻在規律地續期票據的帳號。這一條抓的是長期潛伏，它需要認證日誌的保留期足夠長，否則看不出節律。

寫完這些規則之後，有五件運維上的事決定了它們能不能真的生效。一是保留期：聚合規則需要歷史，保留期短於聚合窗口，規則就永遠只能看到窗口的一部分。二是時鐘：跨機器的聚合必須有統一的時間基準，時鐘不同步會讓關聯規則把因果搞反。三是收集範圍：只收網域控制站不夠，成員伺服器上的本地認證與服務登入同樣要看；只收伺服器不夠，辦公終端上的應用層登入往往才是紅隊用來鋪開的入口。四是白名單治理：白名單是為了壓誤報，但它同時是紅隊的避風港，所以要有定期清理與審批記錄。五是回應：規則產生的告警要有明確的處置動作，否則它只是把事件從一個沒人看的地方搬到另一個沒人看的地方。

9.4 | 這些規則紅隊怎麼繞，以及為甚麼繞不乾淨

紅隊對上面每一條規則都有對應的動作，這些動作在前面各章都出現過：對聚合規則用拉長間隔與分散來源；對新組合規則用沿既有關係走；對服務帳號規則避免用服務帳號做互動式登入；對關聯規則用只認證不落地的

那一輪；對蜜罐帳號規則用先判斷目標是不是活的。這些做法的共同目的是：讓自己的動作落在「這一天本來就會發生的事情」的分佈裏。

但它繞不乾淨，原因是一個結構性的矛盾。紅隊的任務是走出一條以前沒有的路：從外到內、從低權限到高權限、從普通機器到關鍵機器。而前面那些規則看的正是「關係的變化」。也就是說，紅隊要完成任務，就必然要製造新的關係；而新的關係正是這些規則要找的東西。能用來對抗的只是把新關係的數量與速度壓到基線的波動範圍內，而不是讓它不存在。

這個矛盾對兩邊的含義是對稱的。對紅隊：**噪音預算**不是修辭，它是「有多少新關係可以被容忍」的量化上限，而這個上限決定了行動能走多遠。對防守方：偵測憑據這條路的關鍵不在於找到一個完美的閾值，而在於把「帳號與來源的關係」變成一份可以持續維護的資產，因為只有有了這份資產，「新的關係」才是一個可以被自動找出來的東西。

10 | 接着往下走：憑據在域裏怎麼用

本篇講到這裡為止，回答的是「手上有甚麼憑據、它們能在哪裏成立、還能成立多久」。這三個問題的答案合起來，是一張按範圍與時效排序的憑據清單。清單本身不會帶來分數，帶來分數的是拿它去做甚麼。

下一步要回答的是另一個層面的問題：這些憑據在域的認證機制裏是怎麼被使用的。具體包括四件事。域內的認證以 [Kerberos](#) 為主，另有舊的挑戰回應機制並存，它們各自接受甚麼形態的憑據；票據這條路具體怎麼走，從一張票據能拿到甚麼；拿到憑據之後怎麼用目錄查詢把整張圖補全（誰是管理員、誰能登入哪台機器、哪台機器有甚麼服務）；以及從一台普通機器到 [域與域控](#) 之間還隔着哪幾道門。

這四件事都不在本篇展開，它們是第五篇《域與內網橫向》的內容。兩篇的分工是：本篇負責「取得與判斷」，那一篇負責「使用與擴展」。本篇結尾提出的每一份憑據，在那一篇裏都會被當成一個起點，而不是一個結果。

最後再重複一次前提。本篇與第五篇寫的所有動作，都只在拿到授權的演習環境裏成立。技術上的可行性從來不是做這件事的理由，那一紙授權才是。

11 | 術語表

本表是全文的概念登記區，按閱讀順序排列：基礎概念在前，建立在它之上的概念在後。正文裏每個加下劃線的名詞都指向本表的詞條；讀完一條要回原處，用各自環境的返回鍵即可（瀏覽器或 PDF 閱讀器通常是 Alt+左箭頭）。

11.1 | 護網演習

護網演習是由公安部牽頭組織的實戰化網絡攻防演習，用來檢驗重點單位在真實攻擊下的安全防護水平。參演單位以央企、國企、金融、能源、運營商、醫療、教育等重點行業為主。演習按組織層級分國家級、省級、市級，部分行業也會自行組織同類演習。

它與普通滲透測試的區別有三處。攻擊由多支隊伍同時發起、持續多日、事先不打招呼；攻防雙方都被計分，防守方的處置速度直接影響成績；攻擊目標由組織方指定，不由乙方自己劃定範圍。

組織方一般提前約三週通知參演單位。通知內容包括參演單位要防守的目標系統清單，以及演習的起止時間。

11.2 | 紅隊

紅隊是 **護網演習** 中的攻擊方。紅隊的任務是在演習期內攻下分配給自己的目標，拿到可驗證的證據並提交，以此換取分數。

參加演習的紅隊主要有兩類來源：組織方抽調的專業隊伍，以及參演單位或安全廠商派出的隊伍。一支小隊通常三到五人，常見的分工是一個管情報、一個管突破、一個管內網與收尾。

11.3 | 藍隊

藍隊是 **護網演習** 中的防守方，由參演單位自己的人員和它聘請的安全廠商工程師組成。藍隊的任務有三件：及時發現 **紅隊** 的攻擊、及時處置、並把攻擊鏈還原清楚。三件任缺一件，處置的效果都要打折。

演習期間藍隊二十四小時值守。攻擊不會因為提交窗口關閉而停止，只是成果被留到第二天白天再提交。

11.4 | 紅藍對抗

紅藍對抗指紅隊與藍隊分開建制、各自計分的對抗形式。對抗全程有第三方在場。組織方負責提供演習平台、審核紅隊提交的證據、記錄雙方的得分與扣分。裁判的裁定在演習中就是最終結果，所以證據的完整性與可核驗性，在紅隊的工作裏和技術突破同樣要緊。

11.5 | 靶標

靶標是演習中紅隊要攻下的目標系統。每支紅隊有分配好的固定目標。組織方另設一批公共目標放進目標池，供所有隊伍爭奪，因此演習一開始，熱門的公共目標往往同時被好幾支隊伍盯着。

靶標一般由參演單位自報，或由組織方指定。紅隊拿下靶標並提交可驗證的證據，經審核通過才算得分。

「拿下」在一份具體的演習規則裏有明確定義，它通常不是「能訪問這個系統」，而是「能以可驗證的方式在靶標系統上執行操作」。紅隊要

達成的具體目標因此有三種常見形態：在靶標系統上執行一條指定的命令、取得該系統上指定的資訊、或者取得該系統的管理權限。三種形態的分值不同，而判斷有沒有達成的依據是提交的證據，不是紅隊自己的描述。

11.6 | 演習規則

演習規則由組織方在演習開始前下發，規定計分方式、證據提交窗口與紅線。以紅隊為例，常見的規定是提交窗口在白天若干小時內開啟，證據要能證明權限真實，並且確實來自靶標系統。

紅線是規則裏最要緊的部分：禁止拒絕服務攻擊、禁止刪改業務數據、禁止影響生產系統運行、禁止訪問與演習無關的真實個人數據。越過紅線的隊伍會被扣分，嚴重的直接取消成績。

11.7 | 授權

授權是參演單位或組織方給紅隊的書面許可，寫明可以攻擊的目標範圍、可以使用的時間窗、以及允許造成的影響程度。授權的具體內容由演習規則與組織方的通知共同限定。紅隊的一切動作都以此為邊界。

邊界情形有兩種要分清。一種是授權寫明了範圍，但某一台機器的歸屬有疑問；另一種是歸屬沒問題，但某個動作的影響程度接近上限。兩種情形的處理方式相同：停下、向組織方書面確認、留存回覆。紅隊的通行做法是一律以書面為準，因為口頭同意事後無法證明。

11.8 | 邊界設備

邊界設備指部署在單位網絡與外部網絡之間、用來控制或檢查進出流量的設備。常見的有防火牆、遠端接入設備、入侵偵測與入侵防禦設備、郵件閘道、以及負載均衡。

邊界設備在演習裏的價值有兩重。一是它們自己有管理面，管理面一旦被拿下，紅隊等於直接站在內網門口；二是它們的策略配置本身就是情報，讀一份防火牆規則就等於讀懂目標的網絡分區。

補丁落後是這類設備的常態，成因在正文打點那一章展開。

11.9 | 堡壘機

堡壘機是集中管理運維權限的跳板伺服器。管理員不直接連目標伺服器，而是先登入堡壘機，再由堡壘機轉發到目標，所有操作在堡壘機上留一份會話記錄。

它同時是防守方的審計點與紅隊的高價值目標。堡壘機上通常存着大量被管目標的憑據，拿下它等於一次拿到一整批機器的訪問權；反過來，堡壘機會話記錄是藍隊復盤攻擊路徑時最完整的一份材料。

11.10 | 攻擊面

攻擊面指目標單位暴露在互聯網上、能被外部直接觸達的一切資產與入口。它包括域名與子域名、公網 IP 段、開放端口、對外發佈的 Web 應用與接口、遠程接入通道、以及託管在雲上的服務。

攻擊面裏最危險的部分，是沒有人認領的資產：測試環境、演示系統、開發人員自己搭的服務、上線時登記過而後無人維護的舊後台。它們通常不在運維的巡檢清單上，卻和生產系統連在同一個網絡裏。

11.11 | 內網

內網指目標單位內部、不直接暴露在互聯網上的網絡。辦公網、生產網、管理網都屬於內網。[攻擊面](#)是牆外的部分，內網是牆內的部分，兩者由邊界設備隔開。

內網裏的機器通常不能直接訪問互聯網。[紅隊](#)進入內網之後，原先的攻擊機就聯繫不上目標了，必須另找通道把流量送出來。

11.12 | 信息收集

信息收集是[紅隊](#)在動手之前對[攻擊面](#)的系統性摸底，也叫偵察。它包括找出目標到底有哪些資產、這些資產分別是甚麼、由誰維護、有沒有已知的問題。

信息收集貫穿整個演習。演習開始前做的是外圍情報；拿到[攻擊面](#)上的一個入口之後，還要對[內網](#)再做一遍同樣的事。

11.13 | 影子資產

影子資產指單位自己不完全掌握、或根本不知道存在的對外服務。常見來源有四類：開發人員的測試環境、業務上線時臨時開放而後無人維護

的入口、移動端與小程序的後端介面、以及併購與分支機構帶進來的舊系統。

這類資產的共同點是沒有人為它負責，所以沒有人給它打補丁、看日誌、做加固。它們是 [攻擊面](#) 上價值最高的部分。

11.14 | 憑證透明度日誌

憑證透明度日誌是公開的、只可追加的 TLS 憑證簽發記錄。憑證頒發機構每簽發一張公開憑證，通常都要把它提交到一個或多個日誌，日誌對任何人不設讀取條件。規範⁹

對紅隊來說，這份日誌的用處是它記着一張憑證裏寫過的所有域名。一次簽發往往把一批相關域名同時寫進記錄，所以查一個已知域名，常常能順出若干個原本不知道的子域名。

它的盲區有兩個：只用自簽憑證、不對外服務的系統不會出現在日誌裏，以及簽發時間早於日誌上線的憑證也不在其中。所以它要和被動 DNS 一類來源互相印證。

⁹見 [RFC 9162](#)。

11.15 | 被動 DNS

被動 DNS 指長期記錄域名解析結果的歷史數據庫。它不主動查詢，只是把各處觀察到的解析記錄存下來，所以能查到某個域名過去指向過哪些位址。

對紅隊的用處是找已經下線但仍有記錄的資產，以及判斷兩個域名的位址是否曾經重合。它與憑證透明度日誌的盲區正好互補：前者管用過的名字，後者管用過的位址。

11.16 | 自治系統號

自治系統號是互聯網路由體系裏標識一個網絡的編號，英文縮寫是 ASN。一個單位、一家運營商、一家雲廠商都可能擁有若干個自治系統號，每個號對應一組可以整體查詢的 IP 段。

它的用處是把「一個域名」還原成「一片網段」。同一個自治系統號下的位址通常屬於同一個組織，所以紅隊可以用它推斷目標自建機房的範圍，也能用它判斷某台機器是不是目標自己的資產。

11.17 | 指紋

指紋是資產在技術上留下的可識別特徵。信息收集走到一定程度，紅隊就能說出對面這台機器裝的是甚麼操作系統、甚麼中間件、甚麼框架、甚麼版本，甚至能猜到它是哪個廠商的哪款設備。這一組判斷就是指紋。

指紋的用處是選武器。知道指紋之後，紅隊才知道該翻哪個組件的已知漏洞，不必把所有漏洞都試一遍。

11.18 | 網頁應用防火牆

網頁應用防火牆是檢查 HTTP 請求解析構並攔截攻擊特徵的設備或軟件，英文縮寫是 WAF。它部署在網站前面，按規則匹配請求內容，命中規則就阻斷。

它對紅隊的意義是改變試探的成本。同一個入口在沒有 WAF 時可以反覆試，有 WAF 之後每次嘗試都可能被記錄並觸發攔截。所以紅隊要先認出目標是否有 WAF、是哪一款，再決定用甚麼節奏試。

認它的方法在正文認指紋那一節。

11.19 | 內容分發網絡

內容分發網絡是把網站內容緩存在各地節點的服務，英文縮寫是CDN。訪問者連到最近的節點，由節點回源或直接作答。

它給防守方帶來一個副作用：從外部解析到的位址是節點的位址，不是目標真實伺服器的位址。所以紅隊若把節點位址當成目標資產去掃，掃到的東西與目標無關，既浪費時間也可能越過**授權**邊界。

11.20 | 弱點

弱點指目標身上可以被利用的缺陷，範圍比漏洞寬。它包括軟件漏洞、配置錯誤、弱口令與默認口令、權限設置不當、以及業務邏輯上的設計缺陷。

在**攻擊面**上找**弱點**靠**指紋**指路。同一個弱點在不同階段的價值並不相同：暴露在互聯網上的弱點能換來一個入口，藏在**內網**裏的弱點能換來一片區域。

11.21 | 反序列化

序列化是把記憶體裏的物件寫成一段位元組流，反序列化是把這段位元組流還原成物件。這個機制在跨程序傳遞資料時到處都用。

它的危險來自一件事：還原的過程往往會按位元組流裏寫的類型資訊去建立物件，甚至呼叫物件自己的還原方法。於是「能控制位元組流」就等於「能影響對方建立甚麼物件、執行甚麼程式碼」。

危險的觸發點通常在資料要跨程序邊界的地方：工作階段資料、快取、訊息佇列、以及遠端呼叫。這些位置有一個共同特徵，就是資料由一端寫入、由另一端還原，而寫入端未必可信。

11.22 | 越權訪問

越權訪問指使用者做了自己權限範圍之外的事。它分兩類。水平越權是同級別使用者之間互相訪問對方的資料，例如把查詢裏的使用者編號換成別人的；垂直越權是低權限使用者做了高權限的事，例如普通帳號存取了管理介面。

它與其他弱點的差別在於，它通常不需要任何技術鏈，只需要改一個參數或直接訪問一個沒有做檢查的位址。所以它在真實系統裏的出現頻率遠高於外界的預期。

11.23 | 服務端請求偽造

服務端請求偽造指紅隊讓目標伺服器代替自己去訪問一個位址，英文縮寫是 SSRF。觸發點是那些由使用者提供位址、由伺服器去抓取的功能，例如圖片抓取、網址預覽、檔案匯入。

它的價值取決於伺服器能訪問到甚麼。若伺服器能訪問內網位址，這個弱點就從一個功能缺陷變成進入 [內網](#) 的入口，這是它被單獨列出來的原因。

11.24 | 社工釣魚

社工釣魚是從人入手取得訪問權限的手法。常見形式是向目標單位的員工投遞偽裝過的郵件，誘使其點擊鏈接、打開附件、或在仿冒的登錄頁上填寫帳號口令。

信息收集 階段拿到的員工姓名、崗位與郵箱命名規則，在這一步直接變成投遞清單。釣魚的價值在於它繞開全部邊界設備。防禦方把防火牆、WAF、入侵檢測都部署在互聯網入口，而釣魚郵件進的是員工的收件箱。

11.25 | 誘餌

誘餌指釣魚投遞裏給人看的那一層東西，包括郵件正文、附件、以及連結指向的頁面。它與**載荷**是兩件事：誘餌決定對方會不會動手，載荷決定動手之後發生甚麼。

誘餌的成敗取決於它說的事情在對方的工作裏合不合理，與技術關係不大。同一份載荷換一個誘餌帶來的成敗差別，通常大於同一份誘餌換一份載荷。

11.26 | 1day 與 Nday

1day 指剛公開、補丁剛發布不久、多數目標還沒來得及修的漏洞；Nday 指已經公開很久、補丁早已發布、但仍有相當數量目標沒有修的漏洞。兩個詞都是紅隊內部的叫法，不是嚴格的學術術語。

兩者的用法差別在正文漏洞的取捨那一節。要記住的是它們的分界不在漏洞本身，而在目標修沒修。

11.27 | 口令噴灑

口令噴灑指用少量常見口令去試大量帳號，每個帳號只試一兩次。它與逐個帳號窮舉的差別在觸發鎖定的機率。帳號鎖定策略通常按同一帳號連續失敗次數計數，每個帳號只試一次就不會把任何一個帳號試到鎖定。

11.28 | 撞庫

撞庫指拿別處洩漏的帳號口令組合，去試目標單位的系統。它的前提假設是人會在多個系統上重複使用同一套口令。

它與口令噴灑的差別在偵測方式上。撞庫的特徵是大量不同帳號、每個帳號的口令都不同且都失敗；口令噴灑的特徵是大量帳號都試同一個口令。防守方這兩條規則要分開寫。

11.29 | 憑據填充

憑據填充指把已經拿到的憑據拿去試其他系統。它與撞庫的差別在憑據的來源：撞庫用的是別處洩漏的，憑據填充用的是自己在本次行動中取得的。

這一步是內網擴散的主要動力。一次成功的填充就把一台機器的存取權變成一批機器的存取權，所以它的收益遠高於在同一個系統上繼續找漏洞。

11.30 | 多因素認證

多因素認證指登入時要求兩類以上不同性質的憑證，英文縮寫是MFA。不同性質是關鍵：口令加另一個口令不算多因素，口令加一次性的動態碼或一把私鑰才算。分級與要求¹⁰

它對紅隊的直接影響是口令那條路被截斷：知道口令也登不進去。但它並不是不可繞，繞法不在於破解第二個因素，而在於讓第二個因素驗證通過的那一刻被別人利用，詳見正文弱口令那一節。

¹⁰ 見 [NIST SP 800-63B-4](#)，該修訂版在二零二五年發佈，取代了此前的 SP 800-63-3。

11.31 | 打點

打點指在目標系統上取得第一個可控的執行點。它是紅隊在演習裏真正意義上的開始。在打點成功之前，紅隊做的都是準備。

打點通常走三條路：一是攻擊面上存在弱點的 Web 應用與邊界設備，二是社工釣魚換來的員工終端與帳號，三是供應鏈與第三方運維。三條路的目標一致，都是為了拿到一個立足點。

11.32 | 立足點

立足點是打點成功之後，紅隊在目標系統上獲得的那個可控的執行點。表現形式可能是服務器上的一個後門文件、一段注入到正常進程裏的代碼、也可能只是一個能用的帳號。

立足點有三個必須解決的問題：能不能執行命令、能不能穩定存活、能不能反覆回來。只滿足第一條的立足點往往活不過半天。

11.33 | 載荷

載荷是紅隊投放到目標系統上、由目標自己執行的那段內容的統稱。它是一個泛稱，具體分三層：**誘餌**那一層給人看，**加載器**那一層負責把東西弄起來，**被控端**那一層長期留在機器上聽**命令與控制**指揮。

三層的壽命不同，誘餌投遞一次就廢掉，加載器通常在數小時內被端點防護記住，被控端要活到演習結束。所以三者的投入與更新節奏也不同。

11.34 | 加載器

加載器是**載荷**裏第一個跑起來的那一段。它的任務有三件：判斷自己在甚麼環境裏、把真正幹活的內容還原出來、以及換一個已經被信任的宿主去執行它。

它與**被控端**的分界在於是否長期存在。幹完就消失的是加載器；留下來維持**命令與控制**通道的是被控端。

它的痕跡落在終端上，而且是最容易被記錄下來的那一層，所以它也是三層裏最需要反覆更換的一層。

11.35 | 被控端

被控端是 **載荷** 裏長期留在目標機器上、主動回連控制端並接受指令的那一段。它的功能有五件：回連、取指令、執行、回報、以及自我保護。

它與 **立足點** 的關係是：立足點描述的是紅隊在目標上有一個可控的執行位置，被控端是維持這個位置最常見的實現。後續的取 **憑據**、**提權** 與 **橫向移動** 都從它開始。

它的可見性與存活期成反比：要在目標機器上長期存在，就必須有某些東西是持續的，而持續的東西會在日誌或基線裏形成穩定的痕跡。

11.36 | 代碼簽名

代碼簽名指用一個由受信任機構簽發的證書，為程式檔案附上一段可驗證來源的簽章。作業系統與終端防護會據此調整對這個檔案的初始判斷，包括信譽評級、使用者看到的警告、以及部分策略的放行。

它只影響檔案層這一關，程式跑起來之後的行為該被發現還是會被發現。所以它的作用是降低進門的阻力，不是進門之後的安全。

簽名的痕跡在證書本身：證書有實名審核、有簽發紀錄，被濫用之後會被吊銷，因此紅隊用哪一類證書是一個要事先想清楚的決定。

11.37 | 沙箱

沙箱指防守方或分析平台用來在隔離環境裏執行可疑檔案、並記錄其行為的系統。它與真實目標環境的差別在於：沒有真實使用者在用、硬體與系統設定較為固定、以及執行時間通常有限。

對紅隊的意義有兩重。一是它會記錄 **載荷** 在隔離環境裏的完整行為，於是同一版載荷用第二次就會被認出來；二是它能被用來排除明顯有問題的載荷，因為在沙箱裏過得了不代表目標上過得了，沙箱裏過不了則幾乎可以確定目標上也過不了。

11.38 | 重定向器

重定向器是 **命令與控制** 架構裏的一台中間機器。**被控端** 連的是它，它再把流量轉給真正處理指令的控制端。

它的作用是讓被發現時的損失變小，而不是讓自己被發現不了：防守方順著流量查到的是這台機器，上面沒有任何東西，處置它只讓紅隊損失

一跳；同時因為被控端記住的位址沒變，紅隊可以在不換載荷的情況下更換後端。

代價是多了一跳，而且它的位址遲早會被查出來。

11.39 | 命令與控制

命令與控制指紅隊與立足點之間傳遞指令和數據的通道，英文縮寫是 C2。立足點只解決「能執行一次」的問題，命令與控制解決的是「能一直指揮」的問題。

一條合格的 C2 通道要能扛住中斷、要能換協議換端口、要能藏進正常流量裏。演習中常見的做法是把 C2 流量混進 HTTPS 或 DNS 查詢，因為這兩類流量在企業網絡裏幾乎不可能被完全封死。

11.40 | 域前置

域前置指在一條 TLS 連線上，讓網絡中間看到的主機名與實際處理請求的主機名不一致。中間設備只能看到加密連線外層的那個名字，所以按名字做的封鎖會失效。

它靠的是大型內容分發網絡同時承載大量客戶域名的特性。這條路的可用性取決於服務商的策略，近年多數大廠已明確禁止，所以它是一種會過時的手法，不是穩定通道。

11.41 | 隱蔽通道

隱蔽通道指把 [命令與控制](#) 流量藏進看起來正常的流量裏。選哪種流量做載體，判斷標準是它在目標網絡裏是否必須放行。常見載體是 HTTPS、DNS 查詢、以及各類雲服務與協作平臺的介面。

MITRE ATT&CK 把這類做法歸在應用層協議與代理兩族，¹¹

11.42 | 心跳與抖動

心跳指被控端按固定間隔回連控制端，用來確認通道還在、順便取回指令。抖動指把這個間隔加上隨機量，讓回連的時間點不呈現固定週期。

為甚麼要抖動：固定週期的回連在流量統計上是一個明顯的節律，很容易被基線模型挑出來；加了隨機量之後，單看時間間隔就沒有可用的特徵。代價是指令的下發延遲變大。

¹¹見 [T1071](#) 與 [T1090](#)。

11.43 | 進程注入

進程注入指把一段程式碼寫進另一個已經在運行的進程，讓它以那個進程的身份執行。常見做法是在目標進程裏分配記憶體、寫入內容、再讓它開一條執行緒去跑。

紅隊用它有兩個目的：一是借用一個本身被信任的進程，二是讓自己的程式碼不出現在磁碟上。它的痕跡主要留在跨進程的記憶體操作與執行緒建立上，端點防護正是據此偵測的。

11.44 | 白利用

白利用指不帶自己的程式，直接用系統自帶的、本來就有正當用途的工具完成動作。腳本宿主、計劃任務工具、遠端管理工具、以及各類系統管理程式都屬於這一類。

這種做法的難點在防守方：被呼叫的工具是系統的一部分，按檔案信譽攔不住。可用的工具清單有公開整理，¹²

¹²見 [LOLBAS](#)。

11.45 | 端點偵測與回應

端點偵測與回應是裝在終端上、記錄行為而不只比對檔案特徵的防護軟件，英文縮寫是 EDR。它記程序的父子關係、跨進程操作、以及命令列內容，所以對白利用與進程注入這類做法比傳統防毒更敏感。

11.46 | 提權

提權指把已經到手的低權限帳號提升為管理員或系統權限。[立足點](#) 往來得並不好看。Web 服務以普通用戶身份運行，拿下的帳號只能訪問一個目錄。提權就是把這個身份換成能讀寫全盤的那一個。

提權成功的直接收益有兩個：一是能動系統配置，二是能讀到本機保存的帳號口令與密碼哈希。後者比前者重要得多。

11.47 | 提權面

提權面指一台機器上一切能讓低權限身份取得更高權限的地方。它的範圍比[弱點](#) 寬，既包括設定錯誤與權限配置不當，也包括本身合法、卻被授予了超出需要的權力的功能。

一條提權面要同時滿足三個條件： 有一個會以更高權限運行的主體，這個主體會去讀取或執行某個東西， 而那個東西當前身份能改。 三個條件缺一個， 這條路就不通。

紅隊清點提權面時按主體的名目走： 服務、 計劃任務、 核心驅動、 定時任務、 以及被授予額外權力的檔案。 這幾類在兩個操作系統上的叫法不同， 結構相同。

11.48 | 服務設定缺陷

服務設定缺陷指 Windows 服務的設定、 或它所依賴的路徑， 落在當前身份可改的範圍裏。 服務通常以系統身份運行， 所以改動服務設定或它所執行的檔案， 等於讓系統身份替紅隊執行。

常見情形有三種： 服務可執行檔的路徑沒有加引號， 而路徑上有一段目錄可寫； 服務可執行檔所在的目錄可寫； 以及服務物件本身的權限設定允許普通使用者改設定。

它與目錄權限那一類同源， 差別在改的對象不同： 這一類改的是服務指向哪一個檔案。

11.49 | 目錄權限缺陷

目錄權限缺陷指一個高權限程序在運行時會去讀取或執行的位置，落在當前身份可寫的目錄裏。程序按名字去找檔案時，若搜尋順序上有一段目錄可寫，紅隊在那裏放一個同名的檔案，程序就會讀到或執行到它。

它與 **服務設定缺陷** 同源，都是高權限主體碰到了低權限身份能改的東西。差別在這一類不必改任何設定，只要那個位置可寫。

判斷的關鍵不是目錄可不可寫，而是程序會不會真的去那裏找。

11.50 | 核心驅動

核心驅動是運行在 Windows 核心態的程式，它的權限與作業系統同級。普通使用者無權載入驅動，所以這一類提權面要靠兩條路繞過去。

一條是用一個已簽章但有缺陷的驅動：簽章讓它通過檢查，缺陷讓它替紅隊做事。另一條是利用核心本身的漏洞，在核心態執行任意程式碼。

它與 **提權面** 裏其他幾類不同。其他幾類是設定被人用錯了，這一類本質上是 **弱點**，需要版本匹配，而且一旦失手會直接把機器帶下線。

11.51 | 令牌

令牌是 Windows 在一個程序登入之後發給它的身份憑證。令牌裏寫着這個身份是誰、屬於哪些群組、以及持有一組甚麼特權，後續每一次操作都憑令牌判斷權限。

令牌分兩種。主令牌代表登入的那個使用者；模擬令牌代表被模擬的另一個身份，一個執行緒可以在模擬狀態下以別人的身份操作。正因為有模擬，拿到一個高權限的令牌就等於拿到了那個身份。

11.52 | 令牌竊取

令牌竊取指把一個高權限程序持有的 **令牌** 據為己有，從而以那個身份操作。

它與 **提權** 的關係是手段與目的。提權的目標之一是拿到更高權限的令牌，而拿到令牌之後往往就不必再做提權動作了。

11.53 | 特殊權限位元

特殊權限位元指可執行檔上的一組標記，最要緊的兩個是 `setuid` 與 `setgid`。一個帶 `setuid` 的檔案被執行時，執行的身份是檔案的擁有者，而不是呼叫它的那個使用者。

系統裏本來就有一些必須帶這個位元的程式，所以問題不在位元本身，而在一個以更高身份運行、又能被使用者影響其行為的程式：它會讀取使用者給的檔案、接受使用者給的參數、或者去呼叫使用者能控制的另一個程式。

它與能力位元那一類講的是同一個問題的兩種做法，差別在位元是全是全無地換一個身份，能力是把高權限拆成一項一項單獨授予。這一類是 **提權面** 的一種。

11.54 | 命令提權工具

命令提權工具指讓指定使用者以指定身份執行指定命令的工具，在 Linux 上最常見的是 `sudo`。規則寫在一份設定檔以及它引入的目錄裏，每一條規則說明誰、可以以誰的身份、執行哪一條命令。

它成為提權面的原因有三處。被允許執行的命令本身能執行別的命令或開一個 shell；規則用了通配符或前綴比對，實際允許的範圍比寫出來的大；以及規則保留了呼叫者的環境變數，而某些變數會被程式讀取。

還有一類設定是不要求口令，它降低的不是權限，是拿到權限的門檻。這一類是 **提權面** 的一種。

11.55 | 定時任務

定時任務是 Linux 按時間表自動執行命令的機制，常見形態有 cron、一次性的 at、以及以服務管理員為基礎的定時器。

它與 Windows 上的計劃任務是同一件事的兩種實現，缺陷的形態也相同：被執行的腳本本身可寫、腳本所在目錄可寫，或者時間表的定義檔本身可寫。

它另有一處設定特有的問題：定時任務執行時的搜尋路徑比互動式 shell 短，腳本裏若按名字呼叫外部命令，在被搜尋的目錄裏放一個同名程式就能把呼叫接過來。

11.56 | 核心漏洞

核心漏洞指作業系統核心裏的缺陷，利用它可以直接取得系統的最高權限。它的成因類型包括核心介面對使用者傳入的參數校驗不足、競態條件、以及記憶體管理上的錯誤。

它與 **核心驅動** 是核心態這一類提權面在兩個系統上的兩個名字。與其他幾類相比，這一類的成本與風險都高：需要版本匹配，利用過程不穩定，失敗的後果往往是機器崩潰或重啓。

所以在紅隊的次序裏，它排在最後，只在其他路走不通時才考慮。

11.57 | 能力位元

能力位元是 Linux 把最高權限拆成一項一項之後的產物。原本只有最高權限與普通使用者兩種身份，有了能力位元之後，可以單獨把某一項權力授予一個程式，例如讀取任意檔案、切換使用者身份、或管理網路設定。

它成為提權面的原因是：能力集裏有幾項單獨就等於完整的控制權，而被授予這些能力的檔案，往往是廠商或運維為了省事加上去的。

它與 [特殊權限位元](#) 解決的是同一個問題，差別在授予的粒度。兩者的風險相同，都是把不需要的權力給了出去。

11.58 | 憑據轉儲

憑據轉儲指把作業系統與相關服務保存在本機的憑據取出來，包括記憶體快取中的身份資訊、本機帳號資料庫裏的口令雜湊、以及各類服務自己存的憑據。

它是 [提權](#) 之後最優先做的動作，因為一台機器上的憑據往往能用在一批機器上。MITRE ATT&CK 把這類做法歸在 T1003，¹³

11.59 | 憑據

憑據指證明身份的東西，包括帳號與口令、口令的哈希值、Kerberos 票據、以及配置文件與腳本裏明文保存的密碼。[立足點](#) 是位置，憑據是鑰匙。

憑據比漏洞值錢，原因有三個。一個漏洞通常只在一台機器上有效，而憑據可以用在目標單位的許多系統上；不少口令在 [內網](#) 裏被多台機器共

¹³見 [OS Credential Dumping](#)。

用；用憑據登錄產生的流量完全合法，防守方很難從協議層面把它和正常運維區分開。

11.60 | 域與域控

域是 Windows 網絡中集中管理帳號與權限的目錄服務，域控是運行這套服務的服務器。在企業內網裏，域控保存着全網的帳號、口令哈希、以及誰能用哪台機器的授權規則。

對紅隊來說，內網裏的域控是最有價值的一台機器。拿到憑據之後，絕大多數橫向移動都要經過域的認證機制。控制域控，等於拿到整個內網的萬能鑰匙。

11.61 | 票據

票據是域與域控環境裏用來證明身份的憑證。使用者通過認證之後拿到一張票據，之後訪問域內資源時出示這張票據，不必每次都重輸口令。

對紅隊來說票據有兩個特點。一是它可以直接被使用，不需要還原成口令；二是它有有效期，過期之後要重新取得，所以拿到票據之後的動作要在有效期內完成。

11.62 | 橫向移動

橫向移動指紅隊從已有的機器出發，用同樣的憑據去控制內網裏更多機器的過程。它是演習中持續時間最長、最能體現技術水平的環節。

橫向移動很少靠新漏洞，多數時候靠的是目標自己的管理工具：遠程桌面、文件共享、遠程過程調用、計劃任務、以及各種自動化運維腳本。這些工具本來就是為了方便管理員批量操作而存在的。

11.63 | 隧道與代理

隧道與代理是紅隊在自己與內網之間搭的一條中轉通道。內網裏的機器通常不能直接訪問互聯網，紅隊的攻擊機也在外面進不來，兩邊需要一個中間人替它們轉發流量。

常見做法是在已經控制的邊緣機器上開一個監聽端口，把內網的流量包進合法的出網請求裏送到攻擊機，或者讓攻擊機的指令通過這條通道分發到內網各處。這條通道通常與命令與控制通道一起維護。隧道打通與否，直接決定紅隊能走多遠。

11.64 | 隔離區

隔離區指介於外網與 [內網](#) 之間、用來放置對外服務的那一段網絡，也叫 DMZ。對外提供服務的伺服器放在這裏，它與真正的內網之間再隔一道策略。

紅隊拿下隔離區裏的機器並不等於進入內網，兩者之間往往還有一次跨區跳躍。這也是為甚麼從 [邊界設備](#) 進去的收益比從對外業務系統進去更高，邊界設備本身常常同時連着兩個區。

11.65 | 噪音預算

噪音預算指一次行動允許在目標那裏留下多少可被發現的痕跡。它是紅隊自己給自己定的約束，不是防守方的概念。

為甚麼需要這個約束：在互聯網上做情報收集時，掃描量大、被發現也無所謂；進到 [內網](#) 之後，一次全網段掃描就可能直接引來處置，把已經拿下的 [立足點](#) 賠進去。所以內網的每一步都要先算這一步要花掉多少噪音、自己還剩多少。

11.66 | 組態管理工具

組態管理工具指用來批量下發設定與執行腳本到大量機器的運維系統。這類工具在設計上就要能在一台機器上對很多機器下命令，所以它天然具備橫向執行的能力。

紅隊在意它還有一個原因：它的設定檔與下發的腳本裏常常直接寫着高權限帳號。拿到這些檔案等於同時拿到橫向通道與憑據。

11.67 | 特權帳號

特權帳號指在域內或在本機上擁有管理權限的帳號，包括域管理員、本機管理員、以及各類服務帳號。

它們是內網階段的首要目標，原因是 **憑據** 讓攻擊者可以在大量機器上活動，而特權帳號本身的活動範圍往往就直接覆蓋一整片網絡。

11.68 | 權限維持

權限維持指 **紅隊** 在已經拿下的機器上留下能長期反覆回來的入口。演習期間 **藍隊** 會不斷排查，被發現的 **立足點** 會被清掉，所以 **紅隊** 在關鍵機器上通常不止留一條路。

常見的落腳點包括新增的本地帳號與域帳號、計劃任務、系統服務、以及各類 Web 中間件的後門。留在 [域與域控](#) 上的入口價值最高，因為它同時意味着整片 [內網](#) 的訪問權。

11.69 | 痕跡清理

痕跡清理指 [紅隊](#) 在收尾時處理自己留下的記錄，也叫打掃戰場。要處理的東西包括上傳的工具與樣本文件、新增的帳號與計劃任務、被修改的配置、以及系統日誌與安全設備日誌裏的相關條目。

清理的目的首先是保住入口，避免 [立足點](#) 與 [權限維持](#) 的線索被 [藍隊](#) 順着查出來。它與計分的關係是另一回事，兩者之間的取捨寫在正文站樁與打掃那一節。

11.70 | 蜜罐

蜜罐指防守方故意放置的、看起來有價值但實際無人使用的系統或帳號。它不對外提供業務，所以任何對它的訪問都是異常的。

它的價值不在攔截，在於發現。一個蜜罐被觸碰，藍隊立刻知道有人已經進來了，並且能拿到攻擊者的手法與來源。對紅隊來說，過於容易拿下本身就是一個應當放棄的信號。

11.71 | 舉證與報告

舉證指紅隊把拿下的權限整理成組織方可以核驗的證據，再做提交。證據通常包括靶標系統上的命令執行截圖、系統信息頁面、以及能證明操作發生在演習時間窗內的記錄。演習規則對提交窗口有明確規定，過了窗口的成果只能留到下一次。

報告是演習結束後交的另一份東西，內容包括攻擊路徑、用到的弱點、以及藍隊在哪些環節本可以攔住卻沒有攔住。紅隊的報告與藍隊的復盤合起來，才是一次演習對參演單位的真正價值。

11.72 | 服務帳號

服務帳號指為了讓某個服務或應用程式登入系統而建立的帳號。它與人工使用的帳號有三個差別：口令通常很長且很少更換，因為換了要同步改設定；它往往在許多台機器上有權限，因為服務要跨機器訪問資源；它的登入記錄看起來不像人，因為它只在服務啟動時登入。

對紅隊來說它是最有價值的一類憑據。拿到一個服務帳號的口令或雜湊，常常等於拿到一批機器的訪問權，而且用它登入產生的日誌與正常服務啟動的時間一致，不容易被單條規則挑出來。

11.73 | 網絡分區

網絡分區指內網按用途與策略寬鬆程度切成的若干段網絡。常見的四個分區是辦公網、生產網、管理網，以及放置對外服務的那一段。

四個分區的分工不同。辦公網放員工終端，是紅隊最容易進入的一段；生產網放承載業務的伺服器，是分數最集中的一段；管理網放運維與管理系統，包括域與域控、堡壘機與組態管理工具；放置對外服務的那一段與內網之間再隔一道策略。

分區之間由防火牆與存取控制策略隔開。策略的寬鬆程度通常與業務的必要性成反比：辦公網到互聯網管得鬆，辦公網到生產網管得嚴，而管理網到其他各區留一條給運維的通道。紅隊在內網裏遇到的阻力，大部分就是這三道策略的寬鬆程度差異。

11.74 | 虛擬化管理平臺

虛擬化管理平臺指集中管理一批虛擬機的系統，它負責虛擬機的建立、遷移、快照與關機。常見的形態是廠商自帶的管理平臺，也有把多台主機組成資源池的集群版本。

它的價值在它管着的那片虛擬機上。拿到管理平臺的管理權限，等於拿到那片虛擬機上所有系統的控制權：可以掛載映像、可以直接讀虛擬磁碟檔案，也可以把任意一台虛擬機重啟成自己控制的狀態。對紅隊來說，這是繞過作業系統層防護的一條路。

11.75 | 備份系統

備份系統指集中保存各系統歷史資料的系統，它按策略把生產系統的資料複製到備份介質上，並保留若干個歷史版本。

它對紅隊的價值有兩個。一是它保存着已經被刪除或已經被修補的資料，包括系統的舊組態與舊憑據；二是為了還原方便，它的存取權限通常配置得比生產系統更寬鬆，一台備份伺服器上常常能讀到幾十上百台機器的資料。

11.76 | 目錄服務

目錄服務指把網路裏的使用者、群組、機器與權限規則集中存起來，並提供查詢與認證服務的系統。[域與域控](#)就是這類服務的一種實現，它的查詢協定叫輕量目錄訪問協定，英文縮寫是 LDAP。

它有兩個對[紅隊](#)重要的性質。一是所有物件都可以被查詢：只要手上有一個能通過認證的帳號，就能把全網的使用者、群組與機器列出來，不需要掃描。二是權限規則本身也存在裏面，所以「誰能對誰做甚麼」是一個可以直接讀出來的資料結構，這比在一台台機器上看本機權限省事得多。

11.77 | 東西向流量

東西向流量指[內網](#)內部機器之間互相訪問的流量，包括伺服器到伺服器、應用程式到資料庫、以及運維工具到被管機器的流量。與之相對的是南北向流量，也就是客戶端穿過邊界訪問伺服器的流量。

[內網](#)裏的東西向流量在設計上就很大，原因有三個。業務本身是分層的，一個請求往往要經過前端、應用、快取與資料庫四五台機器；內網

的信任關係比互聯網鬆，機器之間預設放行的協定與端口比邊界上多；運維動作本身也在產生東西向流量，補丁分發、日誌上送、監控採集都是。

流量大對紅隊有兩重意義。一是隱蔽：在每天成千上萬條連線裏多加幾條，不顯眼。二是通道：內網的橫向手法之所以能用，正因為目標自己每天都在產生同樣的連線。

11.78 | 出網控制

出網控制指單位對內網機器訪問互聯網所做的限制。常見做法是不給內網機器分配公網出口，所有出網流量必須經過代理，由代理按白名單放行位址、域名與協定。

它對紅隊的限制是直接的。立足點拿下來之後，被控機器連不上攻擊機，命令與控制通道發不出去，所以紅隊必須先解決出網這一環。解決這件事的技術統稱隧道與代理。

出網控制的鬆緊決定了通道的成本。只放行 Web 代理的網絡，通道要做成 HTTP 或 HTTPS 形態並能處理代理認證；允許直連某些業務位址的網

絡，可以借用一台有出網權限的機器做跳板；完全不允許出網的網絡，紅隊只能靠人把資料帶出去。

11.79 | 機器帳號

機器帳號指 **目錄服務** 裏為每一台加入域的機器建立的帳號。它與 **服務帳號** 一樣是帳號，但使用者是機器本身：機器用它向域證明自己的身份，並用它去拉取 **群組原則**、訪問被授權的資源。

它的口令由系統自動輪換，週期以天計，所以紅隊很難長期保存一份可用的機器憑據。但機器帳號在目錄裏也有權限，例如修改自己的某些屬性，這一條屬於後文的權限關係濫用。

11.80 | 遠端桌面

遠端桌面指把一台機器的圖形界面投到另一台機器上操作的功能，微軟的實現叫 RDP。它是最直接的橫向通道：拿一組可用 **憑據** 登入，就能以那個身份在目標機器上操作。

它的前置條件比協議本身重要。目標要開啟遠端桌面服務，帳號要有登入目標的權限，而本機管理員預設就有這個權限。所以紅隊在這一步真正要解決的問題，是先拿到目標機器上管理員級別的憑據。

11.81 | 檔案共享

檔案共享指機器之間通過網路讀寫對方目錄的功能，微軟的實現叫 SMB。它在橫向移動裏承擔兩件事：把要執行的內容送到目標機器上，以及讀取目標機器上本來就有的檔案。

後者的價值往往更高。共享目錄的權限配置鬆弛是 [內網](#) 的普遍現象，而讀取共享目錄不觸發任何執行類告警，只是多了一條檔案訪問記錄。

11.82 | 群組原則

群組原則指 [目錄服務](#) 裏集中下發到一批機器的設定，英文縮寫是 GPO。它用來統一下發口令策略、軟體安裝、腳本執行、以及各類安全設定。

它有兩個對 [紅隊](#) 重要的地方。一是它本身是一條合法的橫向執行通道：原則在機器啟動或用戶登入時被拉取並執行；二是它本身可以被修

改，所以拿到足夠權限之後，改一條原則就等於對一批機器下了一道命令。

11.83 | 遠端程序呼叫

遠端程序呼叫指一個程式呼叫另一台機器上某個介面的機制，英文縮寫是 RPC。在 Windows 網路裏它是系統元件之間通信的基礎，許多管理動作本身就是一次遠端程序呼叫。

紅隊 用它有兩個好處。它在流量上與正常的管理呼叫混在一起；呼叫過程由系統元件完成，不需要紅隊自己帶程式進去。代價是它對呼叫方的權限與兩端的版本有一定要求。

11.84 | 計劃任務與系統服務

計劃任務與系統服務是作業系統提供的兩種自動執行機制。計劃任務按時間或事件觸發，系統服務在系統啟動時自動運行。兩者都能指定以哪一個帳號的身份運行。

它們在橫向移動與 **權限維持** 裏同時有用。橫向用它們：在遠端機器上登記一個任務，由系統自己去執行，紅隊不需要保持一條連線。維持也用

它們：登記一次可以長期生效。代價是兩者都在系統的設定清單裏，而清單是防守方會定期核對的東西。

11.85 | 日誌集中

日誌集中指把各類機器與設備產生的日誌送到同一個地方保存與分析，而不是留在各自的本機上。它在 **藍隊** 那裏是橫向移動偵測的前提：一次橫向移動會在三台以上的機器上留下記錄，只看一台看不出全貌。

要集中的日誌有四類：身分驗證相關的日誌，它記錄誰從哪裏登入；系統與服務的日誌，它記錄服務安裝與任務登記一類的變更；網路設備的日誌，它記錄跨區的連線；以及端點上的記錄，它記錄程式的啟動與命令列。另外還有一件常被忽略的事：集中之後要能存得夠久，否則事發當時的記錄早已經被覆蓋。

11.86 | 域樹與域林

域樹指共用同一個命名空間、由上而下分層的多個 **域與域控** 組成的集合。父域的名字是子域名字的后綴，例如子域 a.example.com 的父域是 example.com。

域林指多棵域樹通過 **信任關係** 連成的整體。林是同一個域之外的另一層邊界：同一個林裏的所有域共用一套結構描述與組態，並且自動互相信任。

對 **紅隊** 來說，林決定了「拿到一個域之後還能不能往上走」。單個域的管理員只在本域有權限，而林根域的管理員組對整個林有效，兩者之間往往還差一步。

11.87 | 信任關係

信任關係指兩個 **域與域控** 之間建立的、允許一邊的使用者被另一邊認可的關係。有了信任關係，一個域的帳號才能訪問另一個域的資源。

它有兩個維度。方向分單向與雙向：單向信任裏只有一邊認可另一邊的帳號，雙向信任裏兩邊互相認可。類型分可傳遞與不可傳遞：可傳遞的信任會把認可關係繼續傳給第三方，不可傳遞的信任只在一對域之間成立。

紅隊 要看的是三件事：本域與哪些域有信任關係、方向是哪一邊、以及本域的管理員身份在對面那個域裏有沒有權限。前兩件可以直接從域的組態裏讀出來，第三件要對照對面域的群組成員才能確定。

11.88 | Kerberos

Kerberos 是 **域與域控** 環境使用的認證協定。它的核心思路是引入一個雙方都信任的第三方，由這個第三方簽發一段有期限的憑證，讓使用者與服務不必互相傳遞口令就能確認身份。

為甚麼要這樣設計：在一個有成千上萬台機器的網路裏，若每個服務都自己保存一份使用者口令來做驗證，口令的副本會散落在每一台機器上，任何一台被拿下都會洩漏。Kerberos 把驗證集中到一個服務上，其餘的服務只驗證那一段簽發出來的憑證。

它每一次通信都包含三樣東西：誰是使用者、想做甚麼、以及一段用金鑰加密的內容。後文把這個協定的三步認證過程按網路封包的形式寫一遍。

11.89 | 主體名稱

主體名稱指 **目錄服務** 裏為某個服務註冊的名字，英文縮寫是 SPN。它的形式是服務類型加主機名，例如某台機器上的資料庫服務註冊成 MSSQLSvc/db01。

註冊它的目的是讓客戶端能用名字找到對應的服務，並在認證時指明自己要訪問的是哪一個服務。因為 [金鑰分發中心](#) 按這個名字簽發憑證，所以主體名稱本身是一份可查詢的服務清單，也是後文委派與口令猜測兩類手法的切入點。

11.90 | 金鑰分發中心

金鑰分發中心指 [Kerberos](#) 裏負責簽發憑證的那個服務，英文縮寫是 KDC。在 [域與域控](#) 環境裏它由域控擔任，所以域控同時是身分資訊的保管者與憑證的簽發者。

它保存兩類金鑰。一類是每個帳號的金鑰，由帳號口令推導出來；另一類是它自己的長期金鑰，用來簽發它發出的每一段憑證。誰拿到那把長期金鑰，誰就能簽發任意身份的憑證，這是域控被拿下之後果最嚴重的原因。

11.91 | 票據授權票據

票據授權票據是 [Kerberos](#) 認證第一步的產物，英文縮寫是 TGT，也叫通行證票據。它由 [金鑰分發中心](#) 簽發，證明這張票據的持有人已經通過了身份驗證。

它與 **票據** 的關係是總與分。使用者拿到它之後，再用它去換取訪問某一個具體服務的票據。**紅隊** 拿到一張可用的票據授權票據，就等於拿到這個使用者在域內的身份，在這個使用者的權限範圍內可以做任何事，而不需要知道口令。

11.92 | 服務票據

服務票據是 **Kerberos** 認證第二步的產物，英文縮寫是 ST，也叫服務授予票據。它由 **金鑰分發中心** 簽發，內容包括持有人是誰，以及一段用目標服務金鑰加密的資訊。

它只對一個服務有效。訪問另一個服務要另換一張，但換的過程可以由系統自動完成，使用者無感。因為它用目標服務的金鑰加密，所以只有那台服務能讀出其中的內容，中間的機器即使截獲也讀不出來。

11.93 | 票據傳遞

票據傳遞指不還原口令、直接把 **票據** 拿到另一台機器上使用。**紅隊** 在本機讀出票據之後，可以把它注入自己的會話，於是以票據所屬的身份訪問域內資源。

它與拿口令登入的差別在痕跡上。用口令登入會在一次身分驗證裏留下記錄，而注入票據只是讓本機多了一段記憶體中的憑證，只要不去請求新的票據，就不一定產生新的認證事件。這也是為甚麼這種做法在應急處置時不容易被還原出來。

11.94 | 偽造票據

偽造票據指紅隊自己造出一張票據，而不是從目標那裏取得。造票據需要一段金鑰：造服務票據需要目標服務的金鑰，造票據授權票據需要金鑰分發中心的長期金鑰。

兩種偽造的後果不同。前者只能訪問一台機器上的那一個服務；後者可以在整個域裏冒充任何人，包括域管理員，而且因為偽造出來的票據與域自己簽發的形態一致，服務端無法區分它與真票據。這一條也解釋了為甚麼域控的長期金鑰一旦洩漏，單靠改口令無法補救。

11.95 | 票據續期

票據續期指在不重新輸入口令的前提下延長票據的使用期限。票據授權票據有兩個時間：一個是單次使用期限，到點之後必須換新的；另一個是整體最長期限，超過之後無論怎麼續都要重新認證。

續期的做法是拿手上的票據再換一張新的票據授權票據。系統在後台自動完成這件事，條件是原票據還在最長期限之內。對紅隊來說，這決定了偷來的票據能用多久，也決定了為甚麼要在拿到票據之後盡快把權限固定下來。

11.96 | 安全識別符

安全識別符是目錄服務給每一個使用者、群組與機器分配的編號，英文縮寫是 SID。它與名字無關：同一個帳號改了名字，SID 不變，所以權限判斷一律按 SID 走，不按名字走。

這一點在紅隊那裏有兩個用途。一是判斷兩個帳號是不是同一個身份：名字可以重複，SID 不會；二是理解權限繼承：一個群組被授予的權限，是通過群組的 SID 寫進存取控制清單的。

11.97 | 權限關係濫用

權限關係濫用指不利用漏洞、只利用目錄服務裏配置得過寬的授權關係來取得更多權限。

可用的關係有四類。一類是 **存取控制清單** 上的細粒度授權，例如某個帳號被授予了對另一個帳號重設口令的權限；一類是群組嵌套，一個看起來普通的群組可能是高權限群組的成員；一類是 **委派** 與 **憑證服務** 的設定；一類是 **機器帳號** 與 **服務帳號** 被授予的權限。

這類手法的特點是每一步都合法，所以流量與日誌上沒有可直接匹配的異常特徵，防守方能依靠的只有權限變更本身的記錄。

11.98 | 存取控制清單

存取控制清單是掛在 **目錄服務** 物件上的一張權限表，英文縮寫是 ACL。表裏每一行叫一個存取控制項目，寫着「哪一個 **安全識別符** 對這個物件能做甚麼」。

它能表達的權限比讀與寫細得多，例如重設口令、修改群組成員、複製目錄變更、以及對某個物件的單一屬性單獨授權。這些細粒度授權是 **權限關係濫用** 那一類手法的基礎。

11.99 | 委派

委派指把以使用者身份訪問另一個服務的權限交給一個服務，讓它在替使用者辦事的過程中能繼續以使用者的名義訪問下游資源。

它常見於三層結構的業務系統。使用者訪問前端的網站，網站再去訪問後端的資料庫；若資料庫按使用者身份做權限判斷，網站就必須拿到使用者的身份才能查。

它按限制程度分三類。無限制委派允許被委派的服務冒充使用者訪問任何服務；受限委派把可冒充的目標限定在一份清單裏；資源型受限委派把授權決定放在資源那一側，由資源自己聲明允許哪些服務冒充使用者來訪問。三類的可利用性差別很大，判斷方法寫在正文裏。

11.100 | 憑證服務

憑證服務指 **域與域控** 環境裏簽發與管理數位憑證的元件，英文縮寫是 AD CS。它把憑證發給使用者、機器與服務，用來做登入、加密與簽名。

它的地位與 **金鑰分發中心** 平行：兩者都能證明身份，差別在憑證是長期有效的，而且簽發之後由持有人自己保存。對 **紅隊** 來說，它的價值

有兩處：憑證本身可以換成身份；它的範本設定裏常常藏着可以讓非管理員申請高權限憑證的配置。

11.101 | 基線

基線指防守方事先記錄下來的正常狀態，包括哪些帳號在哪些時間從哪些機器登入、哪些程式在哪些機器上運行、以及各類服務每天產生多少流量。

它的作用是提供比較的對象。在單條日誌上，一次異常橫向登入與一次正常運維登入幾乎沒有差別；能把它們分開的只有這次行為與基線的偏離程度。所以沒有基線的單位，即使把日誌都收齊了，也很難在演習期間做出判斷。

11.102 | 帳號鎖定

帳號鎖定指系統在一段時間內拒絕某個帳號的認證，即使口令是對的。觸發條件通常按同一帳號連續失敗的次數計算，配套的三個參數是閾值、鎖定持續時間、以及計數重置窗口。

對 **紅隊** 來說它是 **口令噴灑** 這條路上最主要的限制， 因為它把每個帳號的嘗試次數變成一個有限額度。兩個性質決定了預算怎麼花。一次成功的認證會把失敗計數清零； 計數只在重置窗口內累加， 所以兩次嘗試之間隔開一個窗口就不會累加。

11.103 | 暴力破解

暴力破解指在同一個帳號上反覆嘗試大量口令。 它與 **口令噴灑** 的方向相反： 噴灑把嘗試次數攤在很多帳號上， 暴力破解把嘗試次數集中在一個帳號上。

它的直接後果是必然觸發 **帳號鎖定**， 而且會在認證日誌上留下同一帳號的密集失敗。 它划算的場合只有兩種： 目標沒有鎖定策略， 或者手裏已經有口令的雜湊值， 可以拿回本地離線嘗試而不產生任何網路流量。

11.104 | 口令策略

口令策略指一個系統對口令提出的要求， 包括最短長度、 必須包含的字元類別、 最長使用期限、 以及對歷史口令的限制。 與它配套的是失敗處理規則， 也就是 **帳號鎖定**。

對 **紅隊** 來說它要先反推出來， 因為它決定了甚麼樣的口令有可能存在。 反推的來源按可靠性排， 是前端校驗的代碼、 認證界面的報錯訊息、 以及已控機器上的本地策略。

11.105 | 字典

字典指一份用來猜口令的候選清單。 它與別處洩漏來的帳號口令組合不同， 因為它是紅隊按目標的資訊自己生成的。 **口令噴灑** 與 **暴力破解** 都直接吃字典， 所以字典的質量決定了這兩條路的命中率。

按目標單位的公開資訊訂製的字典， 效果遠好於通用字典。 原因是通用字典裏最常見的那些口令， 單位的 **口令策略** 多半已經把它們排除在外； 而由本單位人名與年份拼出來的口令， 看上去既複雜又符合規則。

11.106 | 鍵盤路徑

鍵盤路徑指按鍵盤上相鄰鍵的走向取出來的一串字元， 例如同排相鄰的橫向序列、 三個豎列的縱向序列、 以及按住換檔之後的符號序列。

它是一類獨立的口令來源，與人名地名無關，所以不會被「不含常見詞」這類規則攔住。它的長度通常在六到十二之間，生成時要把起點在鍵盤上平移幾個位置，因為每個人的習慣起點不同。

11.107 | 默認憑據

默認憑據指出廠或安裝之後從來沒有被修改過的初始帳號與口令，也包括那些本來就沒有認證的管理界面。它與猜的區別是不需要猜，只需要知道對方用的是甚麼產品。

查詢的順序是先靠 **指紋** 認出產品，再去廠商文件、產品映像與公開的默認口令清單裏找對應的值。它在 **紅隊** 的演習裏比在互聯網上更有用，因為內部設備很少被互聯網上的人碰過，廠商出廠的默認值可能已經用了很多年。

11.108 | 一次性動態碼

一次性動態碼指只在一次登入內有效的短數字碼，它是 **多因素認證** 裏最常見的一種第二因素。按產生方式分兩類：一類由伺服器發到使用者的手機號碼上，另一類由使用者手上的程式按共用的種子算出。

兩類的弱點不同。前一類的弱點在號碼可以被轉移或攔截；後一類的弱點在種子可以被釣魚頁面即時取走，因為使用者把它輸出來的那一刻，它就在對方手裏。

11.109 | 硬體金鑰

硬體金鑰指一個實體的認證裝置，它是 **多因素認證** 裏目前公認最難繞的一種第二因素。難繞的原因在於它綁定的是來源：認證時把當前網站的名稱放進簽名裏，所以在一個不屬於真實網站的頁面上拿到的簽名，在真實網站上驗不過。

它與 **一次性動態碼** 的差別就在這裏。動態碼證明的是「手裏有東西」，硬體金鑰證明的是「在這個網站上手裏有東西」。後者把認證與被訪問的站點綁在了一起。

11.110 | 中間人釣魚

中間人釣魚指在受害者與真實網站之間放一個透明代理，讓受害者在真實網站上完成一次真實的認證，而攻擊者從中取得認證完成的結果。

它繞過的不是第二個因素，而是第二個因素與後續訪問之間的綁定。前提有四條：使用者在不屬於真實網域的頁面上輸入憑證、代理能通過目標站點的反自動化檢查、目標發出的結果是可轉移的、以及這次認證的結果有足夠長的有效期。

11.111 | 會話憑證

會話憑證指使用者通過認證之後，網站發給瀏覽器用來代表這個身分的東西。它通常是一個工作階段識別碼，放在瀏覽器的儲存區裏，之後的每一次請求都帶上它。它本身就是一個已經通過認證的身分，所以在有效期內不再需要口令與第二因素。

中間人釣魚 要拿的就是它。圍繞它的兩個問題決定了風險大小：它與來源或裝置綁不綁定，以及它的有效期有多長。

11.112 | 身分憑證

身分憑證指由某個頒發機構簽發、用來證明主體身分的電子證書。驗證的一方不檢查口令，只檢查這張憑證上的簽名是不是來自它信任的頒發機構。

所以只要簽發用的私鑰被別人拿到，就可以造出任意主體的身分憑證，整個口令與第二因素的環節都被跳過去。它不參與口令輪換，所以在口令這條路被切斷的環境裏是一個獨立的出口。它在域環境裏的展開屬於內網與域那一篇。

11.113 | 記憶體轉儲

記憶體轉儲指把某個程序佔用的記憶體整塊取出來，再從中解析出需要的內容。在憑據這一條路上，它針對的是負責認證的那個程序，因為互動式登入留下的口令材料、雜湊與票據都在那塊記憶體裏。

它是 **憑據轉儲** 裏收益最高的一步，需要在 **提權** 之後才有權限做。它也是 **端點偵測與回應** 最關注的一類行為，因為正常運維裏幾乎不會有程序去讀認證程序的記憶體。

11.114 | 口令雜湊

口令雜湊指口令經過單向運算之後得到的值。系統保存它而不是保存口令本身，認證時把輸入的口令算一遍再比對。這個值也被叫做哈希，是同一個東西的兩種叫法。

它的用途有兩個：可以用在那些以雜湊本身作為認證材料的協定上，也可以通過運算還原成明文。它的限制是不能用來改口令，而且在目標關掉了舊認證協定的環境裏完全失效。

11.115 | 快取憑據

快取憑據指機器替 **域與域控** 裏的帳號保存下來的登入材料，它存在的理由是讓使用者在 **內網** 裏的認證伺服器不可達時仍能登入。它保存的數量有上限，預設是最近登入過的十個帳號。

它與本機帳號的雜湊的差別在範圍。本機帳號的雜湊只對這一台機器有效，而快取憑據可以用來推導那個域帳號的口令，推出來之後範圍就是整個域。它的強度比本機雜湊高，所以它是本地取得憑據裏最值得花算力的一種。

11.116 | 離線破解

離線破解指把取到的 **口令雜湊** 拿回自己的機器上運算，把口令還原成明文。它與在線猜的差別在於它不產生任何網路流量，所以不受 **帳號鎖定** 與速率限制的約束。

它的代價是需要算力，而且能不能還原取決於演算法的快慢與口令的熵。同一個弱口令在快演算法下幾秒就出來，而一個足夠長的口令在慢演算法下是算不出來的。

11.117 | 本機帳號

本機帳號指只存在於某一台機器上的帳號，它不在域的目錄裏。它的範圍就是這一台機器，所以拿它的 **憑據** 對 **橫向移動** 沒有幫助。

它的價值在於讀取這台機器上更多的東西，以及作為一個不起眼的落腳身分。判斷一個帳號是不是本機帳號，看它有沒有網域前綴、以及它在本機的帳號清單裏能不能找到。

11.118 | 本機管理員

本機管理員指某台機器上擁有管理權限的本機帳號。它的範圍是「所有共用同一組本機管理員口令的機器」，這一批有多大取決於部署方式。用統一映像部署、或者用 **組態管理工具** 統一下發口令的環境裏，這一批可能覆蓋整片辦公網；而逐台手工裝出來的環境裏，每一台的口令都不一樣。

判斷這一批有多大，最直接的方法是比較多台機器上同一個 **本機帳號** 的雜湊是否相同。用它去登入一批機器在防守方眼裏動靜很大，所以合理的用法是只挑其中少數幾台關鍵的。

11.119 | 域帳號

域帳號指存在於域的 **目錄服務** 裏、可以在域內使用的帳號。它的範圍是整個 **域與域控** 管轄的範圍，具體能到哪一步取決於它屬於哪些群組。

它是四級憑據範圍裏價值最高的一級，也是痕跡最重的一級：每一次用它做認證都會留下一條記錄，而這些記錄在多數單位裏是被集中收集的。它與 **服務帳號** 的差別在使用方式而不在權限大小：服務帳號的動作發生在後台，域帳號的動作往往伴隨一次明確的登入。

11.120 | 認證日誌

認證日誌指記錄「誰在甚麼時間從哪裏認證到哪裏」的那一類日誌，它與記錄程序行為的主機日誌是兩套東西。它的內容包括登入成功、登入失敗、取得票據、以及帳號被鎖這幾類事件。

它與主機日誌的差別決定了偵測的難點。主機日誌的單條記錄可以單獨判斷；而認證日誌的單條記錄在一個每天有幾十萬條同類記錄的環境裏幾乎沒有資訊量，只有把它聚合成「帳號與來源的關係」、再與 [基線](#) 比較才有意義。它也是 [痕跡清理](#) 最難處理的一類記錄，因為它不在被入侵的那台機器上，而在認證伺服器與 [日誌集中](#) 的平台裏。它與 [蜜罐](#) 配合使用時訊號最乾淨。